

Verified Semantic Probes for JavaScript Interpreters

*Operational and denotational foundations, runtime verification, static analysis,
proof infrastructure, and a goja research workbench*

Research synthesis and technical design document

Prepared for the go-go-goja / goja instrumentation project

July 2026

LIMITATION / FAILURE MODE | Status of the document

This is a research-grounded architecture and semantics proposal, not a completed soundness proof or a claim that an implementation already exists. Formal claims are stated as proof obligations or research hypotheses unless a cited work establishes them for its own system.

ABSTRACT

This document develops a theoretical and practical account of adding an eBPF-like, dynamically attachable, verified probe layer to a JavaScript interpreter, using goja in Go as the concrete target. The proposal is not to reproduce the Linux eBPF instruction set. It is to retain the deeper architecture: stable attachment points, a restricted and statically checked probe language, typed state maps, bounded helper operations, detachable links, and a control plane that keeps the interpreter hot path small. The critical change is that attachment points are defined at language-semantic boundaries rather than only at implementation functions or machine instructions.

The document connects that architecture to structural operational semantics, labelled transition systems, abstract machines, trace semantics, monads, algebraic effects, interaction trees, runtime verification, parametric monitoring, contracts, abstract interpretation, symbolic execution, counterexample-guided refinement, hyperproperties, proof-carrying code, typed assembly language, software fault isolation, executable language specifications, and verified compilation. It then turns those connections into concrete design requirements for goja: an event ABI, a source-to-VM semantic map, explicit observation capabilities, a probe verifier, proof-relevant traces, and integration with go-go-goja runtime ownership and initialization.

The practical objective is a daily academic workbench. A researcher should be able to state a candidate semantic rule or invariant, attach it to compiled semantic sites, execute generated or real workloads, compare the resulting trace against an executable model, minimize divergences, export obligations to an SMT solver or proof assistant, and preserve a replayable certificate. Runtime evidence cannot replace universal proof; however, a carefully designed semantic event layer can make formal models executable against a real interpreter, expose incorrect assumptions early, and support residual verification in which static proofs and runtime monitors share one property language.

DESIGN CONSEQUENCE | Central design thesis

The interpreter should expose a versioned labelled transition interface at semantic boundaries.

Probes should be verified interpretations of those labels, not unrestricted JavaScript callbacks.

Correctness depends first on the event mapping and only second on the safety of the probe VM.

The same probe IR should admit concrete, symbolic, abstract, and logical interpretations with separately stated soundness relations.

Claims, non-claims, and terminology

Table 1. Interpretation of the document's main claims.

Statement	Status in this document
A semantic event layer can be implemented in goja.	Engineering claim supported by the interpreter structure; requires a fork or upstream API and extensive conformance tests.
Read-only probes are transparent.	Desired theorem. It requires an explicit erasure or simulation proof and careful treatment of time, allocation, scheduling, and host effects.
A verified probe proves the JavaScript program correct.	False. The verifier establishes safety and boundedness of the probe; universal program correctness requires a separate proof or sound static analysis.
Passing runtime monitors establishes a property for all executions.	False in general. It establishes only that no violation was observed on monitored executions, unless combined with a coverage or proof argument.
The event ABI determines an observational theory.	Theoretical claim: giving probes more events makes more implementations or programs distinguishable.
The eBPF analogy is exact.	False. The useful analogy concerns safe extension architecture; JavaScript semantic probes need a different event model, state interface, and verifier.

Terminology

- Semantic event: a typed label representing a source-language, specification-level, or deliberately implementation-level transition.
- Probe: a restricted program attached to selected event sites and executed with bounded, typed observations.

- Monitor: a stateful interpretation of an event stream that reports acceptance, violation, or an inconclusive status.
- Semantic map: metadata relating source spans, AST nodes, compiler sites, VM program counters, specification operations, and formal-rule identifiers.
- Proof-relevant trace: an execution record containing enough bounded witness data for an independent checker to validate selected transitions or claims.
- Transparent observation: instrumentation whose events and internal monitor state can be erased without changing the chosen observable behavior of the program.

Contents

Part I	Motivation, architecture, and research lineages
1	Problem statement and design thesis
2	The research lineages that meet in semantic probes
3	Research questions and evaluation criteria
Part II	Operational semantics of instrumentation
4	Configurations, abstract machines, and semantic boundaries
5	Labelled transition systems and the event algebra
6	From VM instructions to semantic macro-events
7	Product semantics for runtime monitors
8	Correctness obligations for the event layer
9	Exceptions, constructors, generators, async functions, and jobs
10	Observation, enforcement, and intervention
11	Causality, host semantics, and multi-agent executions
Part III	Denotational and compositional interpretations
12	Trace denotations and infinite behavior
13	Monads, algebraic effects, and event handlers
14	Interaction trees and weak bisimulation
15	Metrics, costs, and folds over traces
16	Observational equivalence and full abstraction
Part IV	Verified probe execution
17	A restricted probe IR and verifier
18	Opaque handles and effect-controlled state projection
19	Maps, frame-local storage, and parametric monitoring
20	Safety, resource bounds, and the trusted computing base
Part V	Static analysis, runtime verification, and proof
21	One probe language with four interpretations
22	Abstract interpretation and abstract machines
23	Symbolic execution, concolic testing, and refinement
24	Contracts, hybrid checking, and residual verification
25	Temporal properties and hyperproperties
26	Proof-relevant traces and independently checked evidence
Part VI	JavaScript, goja, and the academic workbench

27	JavaScript semantics and executable specifications
28	Mapping goja execution to a semantic event ABI
29	Stable metadata and the semantic map
30	Daily academic tools and research workflows
31	Worked example: a ledger contract and protocol monitor
32	Evaluation methodology
33	Implementation roadmap and open research questions

Appendices	Formal core, event schema, probe IR, CLI sketches, and references
-------------------	--

How to read this document

Parts II and III give the mathematical interpretation of semantic probing. Part IV turns those foundations into a safe extension architecture. Part V explains how one property language can support runtime, symbolic, abstract, and theorem-prover workflows. Part VI makes the proposal concrete for JavaScript and goja. Readers primarily interested in implementation can begin with Chapters 17-20 and 28-33; readers primarily interested in semantics can begin with Chapters 4-16 and then return to the implementation chapters.

Citations use an author-date style. References to goja and go-go-goja describe the inspected source snapshots listed in the bibliography. ECMAScript references use the 2026 edition of ECMA-262. Formal notation is intentionally lightweight: the objective is to state interfaces and proof obligations clearly enough that they can be mechanized later in Redex, K, Rocq, Lean, or another framework.

Part I. Motivation, architecture, and research lineages

Why interpreter-level semantic probing is a research instrument rather than merely a profiling feature.

1. Problem statement and design thesis

The problem is to let analyses attach to a running JavaScript interpreter at semantically meaningful boundaries without turning every analysis into privileged, unbounded interpreter code.

1.1 The gap between observability and semantics

Production instrumentation typically observes implementation artifacts: operating-system probes, machine addresses, native call stacks, allocation sites, counters, or log statements. Formal semantics typically describes source-language configurations and transition rules. The two views are useful but poorly aligned. A source-level theorem about abrupt completion, environment lookup, Promise reaction ordering, or constructor return behavior cannot be checked reliably by observing only Go functions or wall-clock traces. Conversely, a formal model that cannot be related to the actual interpreter provides weak diagnostic help when a real execution diverges from the model.

The proposed layer closes that gap by requiring the interpreter to publish a versioned event vocabulary at semantic boundaries. A call event should mean that a call is about to be performed under a defined notion of callee, receiver, argument list, and call-site identity. A function-return event should mean that the function has semantically completed normally, including any finally processing. A function-throw event should mean that an abrupt completion actually crosses the function boundary, not merely that a throw instruction executed. This distinction is the core of the architecture.

The event layer is paired with a restricted probe language. Probes are not arbitrary JavaScript functions and do not receive raw VM pointers. They execute against typed, borrowed observations; use bounded helpers; update typed maps; emit bounded records; and, in an enforcement capability, report a structured invariant violation. This follows the safe-extensibility pattern of packet filters, proof-carrying code, typed assembly language, software fault isolation, DTrace, and eBPF, while changing the event and state model to suit a language interpreter (McCanne and Jacobson 1993; Wahbe et al. 1993; Necula 1997; Morrisett et al. 1999; Cantrill et al. 2004).

A semantic-probe architecture separates execution, events, and interpretations

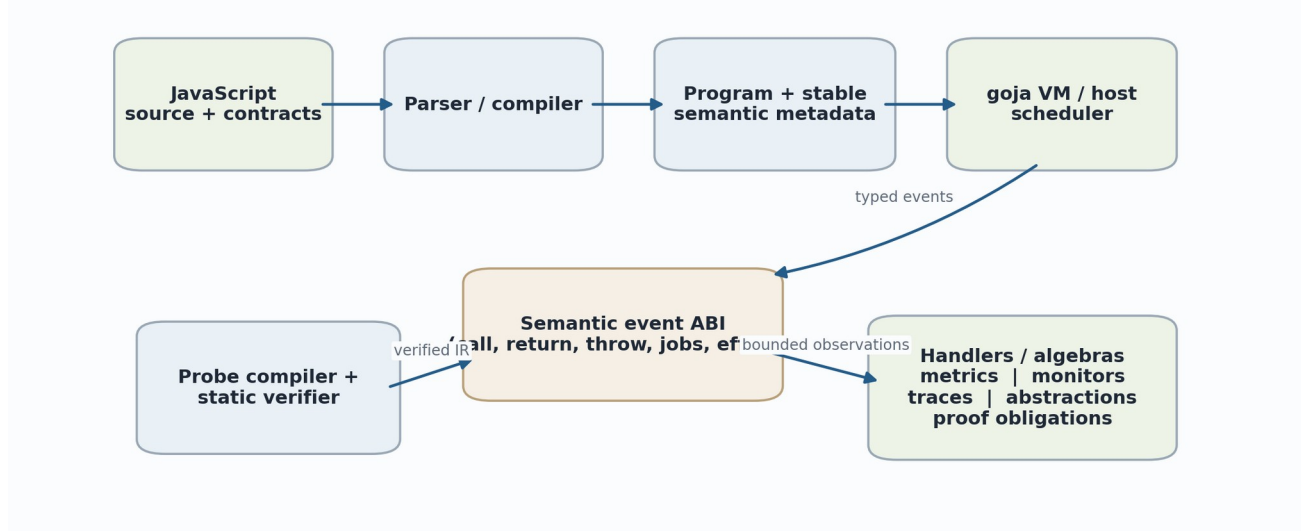


Figure 1. Proposed separation of compilation, interpreter execution, semantic events, and verified interpretations.

1.2 Why an eBPF-like architecture, but not eBPF itself

The useful eBPF analogy is architectural. Linux accepts extension programs only after verification, grants access through a helper interface rather than arbitrary kernel pointers, provides typed persistent maps, attaches programs through

explicit link types, and enforces program-specific execution constraints. The original Berkeley Packet Filter already demonstrated how a small verified machine could safely move filtering work close to a high-volume event source. DTrace later generalized safe dynamic instrumentation around named providers, predicates, aggregations, and low disabled-probe cost (McCanne and Jacobson 1993; Cantrill et al. 2004; Linux Kernel Documentation 2026).

A JavaScript semantic probe layer needs different primitives. The values of interest are not packet bytes or kernel structs but language values, lexical bindings, completion records, object identities, call-site metadata, job identities, Promise dependencies, and source spans. The dominant correctness problem is not only memory safety; it is whether the event corresponds to the intended source-language transition. A verifier can establish that a probe will terminate and cannot corrupt the VM while the event mapping remains semantically wrong. Therefore the event ABI and its refinement relation to the source semantics belong in the trusted base.

1.3 The design thesis

Interpreter execution + semantic event ABI + verified handlers = semantic workbench

The terms are independent components with independent proof and testing obligations.

The design thesis has four parts. First, the interpreter should expose a labelled transition interface rather than a bag of callbacks. Second, the probe language should be small enough to verify, execute predictably, and interpret in several semantic domains. Third, the compiler should produce stable metadata that relates source constructs, VM sites, and formal rules. Fourth, online monitoring and offline analysis should consume the same event records, so that a runtime violation, a differential trace, an invariant-mining result, and a proof-assistant artifact refer to the same execution vocabulary.

DESIGN CONSEQUENCE | Why semantic boundaries matter

Instrumenting only source syntax misses calls introduced by getters, proxies, built-ins, host functions, direct eval, generated functions, and internal specification operations. Instrumenting every VM instruction is complete at the implementation level but too intensional, expensive, and unstable for most semantic claims. Semantic macro-events are an explicit abstraction layer between these extremes.

2. The research lineages that meet in semantic probes

The proposal is best understood as a convergence of safe dynamic extension, runtime verification, contracts and weaving, and executable semantics.

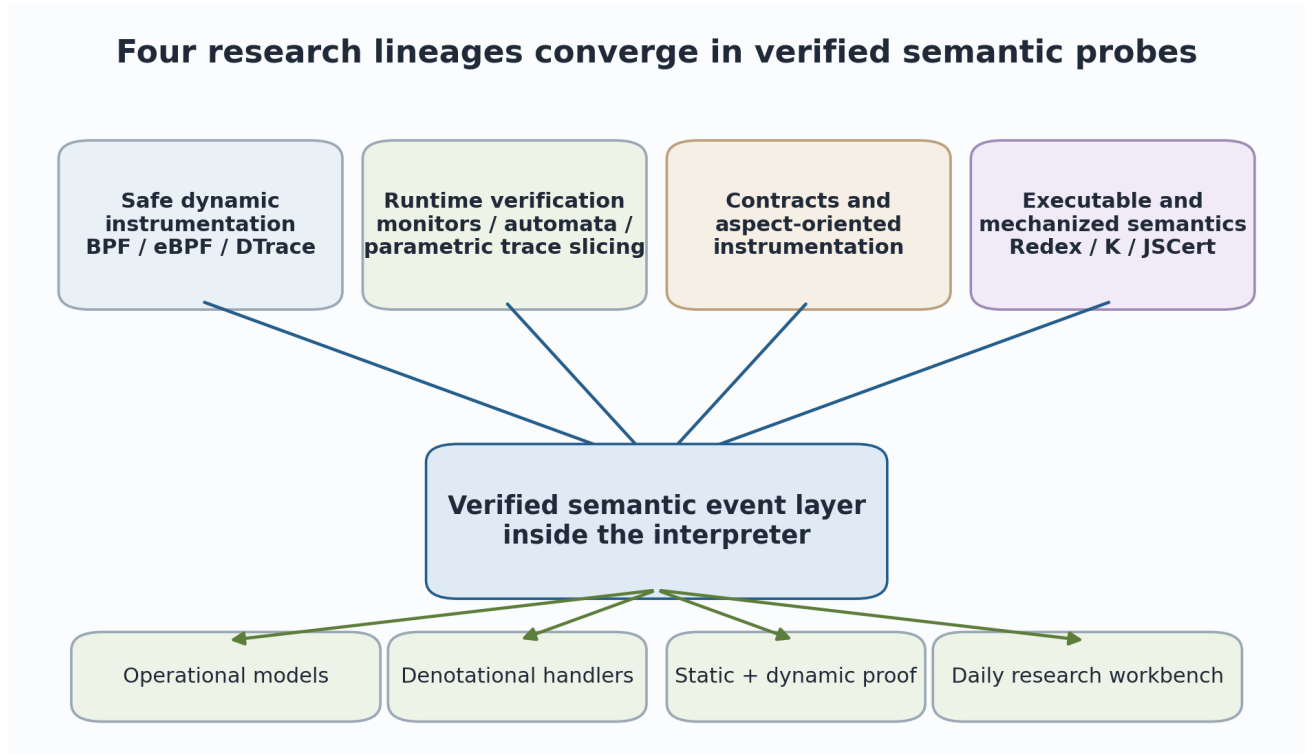


Figure 2. Four research lineages contributing distinct obligations and design patterns.

2.1 Safe dynamic instrumentation

BPF, DTrace, software fault isolation, proof-carrying code, and typed assembly language address a common systems problem: how to admit extension code into a privileged or high-frequency execution environment without trusting the extension author. Their answers differ in mechanism, but the recurring ideas are restricted instruction sets, static validation, explicit capabilities, bounded state, and a small trusted checker. These ideas transfer directly to interpreter probes. They justify a separately verified IR, helper signatures, map schemas, attachment capabilities, and budget accounting. They do not by themselves justify the meaning of an event or the correctness of a JavaScript state projection.

DTrace contributes an additional lesson: stable providers should expose semantic names and typed arguments while hiding implementation details. A DTrace provider is more durable than probing an arbitrary internal function because it declares the intended observability contract. The analogue for goja is a semantic provider whose events are function entry, completion, exception propagation, property effects, job scheduling, and other language operations. As with DTrace, disabled probes should have near-zero effect, and enabled probes should use aggregations rather than forcing every raw event through an expensive external pipeline (Cantrill et al. 2004).

2.2 Runtime verification and monitoring-oriented programming

Runtime verification studies how formal properties are checked against an execution trace. The basic monitor consumes events and maintains a verdict state. Temporal logics, automata, regular properties, and rule systems provide different specification languages; three-valued or multi-valued semantics are often required because a finite prefix may neither prove nor refute a liveness claim (Leucker and Schallhart 2009; Bauer, Leucker, and Schallhart 2010, 2011; Bartocci and Falcone 2018).

Monitoring-oriented programming and parametric monitoring are particularly relevant. Monitoring-oriented programming treats formal specifications as deployable monitors woven into selected program locations. Parametric monitoring handles events carrying object, transaction, request, or resource identities by slicing one global trace into the relevant per-parameter traces before applying a base monitor (Chen and Rosu 2003, 2007; Rosu and Chen 2012; Havelund et al. 2018). Typed maps and stable value identities in the proposed system provide the interpreter substrate for this style of monitoring.

2.3 Contracts, aspects, and dynamic enforcement

Higher-order contract systems establish a disciplined account of preconditions, postconditions, blame, and wrappers around functions. JavaScript contract systems such as TreatJS show that higher-order behavioral contracts can be implemented in a dynamic language while attempting to isolate contract execution from program execution (Findler and Felleisen 2002; Keil and Thiemann 2015). Hybrid type checking and soft contract verification show a second important pattern: discharge what can be proved statically, and retain dynamic checks for residual obligations (Flanagan 2006; Nguyen, Tobin-Hochstadt, and Van Horn 2014).

Aspect-oriented programming contributes the notion of join points and weaving: instrumentation is attached to a structured model of program execution rather than inserted as arbitrary source edits (Kiczales et al. 1997). The proposed semantic event ABI can be viewed as a carefully restricted join-point model. However, the architecture deliberately separates observation from intervention. Advice that changes arguments or return values alters program semantics, while an observational probe should admit an erasure theorem.

2.4 Executable and mechanized semantics

Redex, K, JSCert, KJS, and JSExplain demonstrate different ways to make language semantics executable and useful. Redex integrates reduction relations, random testing, and model exploration into semantics engineering. K supplies an executable semantic framework with labelled rules and derived analysis tools. JSCert mechanizes a substantial JavaScript specification and extracts a reference interpreter. KJS executes a large JavaScript semantics and uses semantic-rule coverage and symbolic execution. JSExplain exposes both the specification-oriented interpreter state and the JavaScript program state in a double debugger (Felleisen, Findler, and Flatt 2009; Rosu and Serbanuta 2010; Bodin et al. 2014; Park, Stefanescu, and Rosu 2015; Chargueraud, Schmitt, and Wood 2018).

These systems start from a semantic model and derive execution or analysis. The goja proposal starts from a concrete interpreter and adds an abstraction interface that can be compared with those models. The methodological bridge is differential semantics: normalize both executions to a common event alphabet, align events by source and causal identity, and stop at the first divergence. The result is not a proof of interpreter correctness, but it localizes model and implementation disagreement much more sharply than final-output comparison.

Table 2. Research ideas inherited by the semantic-probe design.

Lineage	Transferred idea	New obligation in a language interpreter
BPF / eBPF / DTrace	Verified extensions, helpers, maps, attachment links, low disabled overhead	Events must correspond to language semantics, not only memory-safe implementation sites.
Runtime verification	Automata, temporal properties, finite-prefix verdicts, parametric slicing	The event stream must be complete for each declared monitor property.
Contracts / AOP	Join points, pre/postconditions, blame, weaving, residual checks	Observation must be separated from enforcement and intervention.
Executable semantics	Rule labels, reference execution, semantic coverage, differential traces	A semantic map must relate VM transitions to formal rules and specification operations.

3. Research questions and evaluation criteria

The project becomes scientifically useful when its engineering goals are restated as falsifiable semantic and systems questions.

3.1 Core research questions

- Event adequacy: Can a practical interpreter expose events that correspond to a well-defined source or specification semantics without reproducing the entire formal configuration?
- Transparency: Under which observational model can read-only probes be erased? Which quantitative or scheduling observations necessarily invalidate strong transparency?
- Completeness: Can the system guarantee that every transition relevant to an attached monitor is observed exactly once, including exception unwinds, native calls, dynamic code, and async resumptions?

- Safe extensibility: Can a small verifier establish type safety, bounded execution, bounded memory retention, non-reentrancy, and helper capability compliance with low attachment latency?
- Compositional interpretation: Can one probe IR support online execution, symbolic execution, abstract interpretation, and logical translation without conflating their distinct soundness obligations?
- Academic utility: Does semantic tracing reduce the time required to find errors in a semantics, test suite, compiler mapping, or proof invariant compared with ordinary logging and final-output differential testing?
- Performance: Can disabled event families remain close to a predictable branch, while common enabled probes remain cheap enough for interactive and workload-scale use?

3.2 Evaluation dimensions

Table 3. Evaluation plan and principal validity threats.

Dimension	Representative measure	Threat to validity
Semantic fidelity	Agreement with executable model; manually audited event order; rule coverage	Both model and mapping may share the same mistaken assumption.
Probe safety	Verifier rejection tests; fuzzing; resource-budget enforcement	A safe probe can still observe an incorrect event or unsafe native helper.
Transparency	Differential outcomes and traces with probes erased	Timing, allocation, scheduling, and garbage collection may be observably perturbed.
Performance	Disabled, no-op, counter, contract, and trace overhead per event class	Microbenchmarks may not represent host-heavy or async workloads.
Research productivity	Time-to-divergence, minimized counterexample size, model defects found	Tool familiarity and benchmark selection can dominate results.
Reproducibility	Pinned source, event schema, monitor, trace digest, replay checker	Host I/O or nondeterministic scheduling may remain unrecorded.

3.3 Minimal publishable theorem and artifact

A credible first research result should not claim complete JavaScript verification. A narrower result is stronger: define a small source language with calls, returns, exceptions, and constructors; define a corresponding abstract machine and event-labelled semantics; implement those events in a goja subset or fork; prove an erasure theorem for read-only probes in the model; and validate event correspondence empirically on the implementation. The artifact should include the model, event schema, verifier, interpreter patch, randomized tests, semantic coverage report, benchmark suite, and independently replayable traces.

LIMITATION / FAILURE MODE | Recommended claim boundary

The first paper should claim a semantics-preserving instrumentation architecture for a defined event subset, not a verified JavaScript implementation. Broad language coverage can be evaluated empirically while theorem statements remain scoped to the mechanized core.

Part II. Operational semantics of instrumentation

How semantic events arise from configurations and transitions, and what must be proved about their implementation.

4. Configurations, abstract machines, and semantic boundaries

Operational semantics makes the event layer precise by asking what configuration changes, at what granularity, and under which rule.

4.1 Small-step, big-step, and machine semantics

Structural operational semantics defines program behavior through inference rules over configurations. A small-step relation exposes individual transitions; a big-step relation connects an initial phrase and state to a final result; an abstract machine makes control, environments, stores, and continuations explicit. Standard treatments emphasize that the choice is methodological: small-step semantics is convenient for interleaving, divergence, and intermediate observations, while big-step semantics is compact for terminating evaluations (Plotkin 2004; Winskel 1993; Gunter 1992; Nielson and Nielson 2007).

A JavaScript interpreter is naturally described by an abstract-machine configuration that includes an expression or program counter, lexical environment, store or heap, continuation or call stack, exception state, job state, and host state. A simplified configuration can be written as $C = \langle p, \rho, \sigma, \kappa, j, h \rangle$. The actual goja VM contains implementation representations of many of these components, but the ECMAScript specification warns that execution contexts are specification devices and need not correspond to implementation artifacts. Consequently, a semantic map must relate rather than identify the two (ECMA International 2026).

$$C = \langle p, \rho, \sigma, \kappa, j, h \rangle$$

Program point, lexical environment, store, continuation, job state, and host state.

4.2 Semantic boundaries as abstraction points

Not every machine transition should be public. An interpreter may use several instructions to implement one source operation, or one instruction may call into a helper that performs many specification operations. The event layer therefore defines macro-transitions: finite sequences of internal steps that begin and end at stable semantic boundaries. The implementation may change inside the macro-step without changing the event contract, provided the abstraction relation and emitted observations remain valid.

Examples include function invocation, normal function completion, exception propagation across a frame, constructor completion, property read with a specified observation mode, Promise reaction enqueue, job start, await suspension, continuation resume, and module resolution. Each event needs a precondition, a postcondition, a defined observation projection, and a statement of whether it is source-semantic, specification-derived, or implementation-intensional.

WORKING DEFINITION | Macro-event criterion

A semantic macro-event should be stable under reasonable interpreter refactoring, expose enough information for its intended analyses, and avoid exposing intermediate state that the language or chosen formal model does not make observable.

4.3 Continuations and frame identity

Function calls are not adequately modeled as paired callbacks unless the implementation records the continuation created by the call. An interpreted call normally changes the active program, environment, stack base, and continuation; its return is observed only when the callee later restores the caller. Exceptions may discard several continuations at once. Generators and async functions may suspend a continuation and resume it in a later job. Stable FrameID and ContinuationID values are therefore semantic correlation objects, not optional logging conveniences.

The machine view also clarifies a subtle distinction. A call-site event belongs to the caller and measures the invocation expression. A function-entry or function-exit event belongs to the callee. A proxy, bound function, native host function, constructor, or async wrapper can insert transitions between these views. Analyses should attach to the event they actually mean rather than assume that call and entry are identical.

5. Labelled transition systems and the event algebra

A semantic probe layer turns an unlabelled execution relation into a labelled transition system with a typed observation algebra.

5.1 Labelled transitions

Let $C \rightarrow C'$ be the uninstrumented transition relation. Instrumentation refines it to $C \rightarrow[\text{ell}] C'$, where ell is either an internal label tau or a semantic event. Labels permit traces, bisimulation arguments, temporal monitoring, and compositional interpretation. Labelled transition systems are standard in structural operational semantics and concurrency theory; their meta-theory supports congruence formats, simulations, bisimulations, and equivalence reasoning (Hennessy and Milner 1985; Plotkin 2004; Mousavi, Reniers, and Groote 2007; Sangiorgi 2011).

$$C \rightarrow[\text{tau}] C' \quad \text{or} \quad C \rightarrow[\text{event / observation}] C'$$

Internal transitions are hidden; selected semantic transitions carry typed labels.

5.2 Event records are typed propositions about execution

An event record is not merely a timestamped string. It asserts that a transition of a declared kind occurred at a declared semantic site and that each field has a defined relation to the pre-state, post-state, or causal context. For example, FunctionReturn should specify whether Result is the value before or after constructor coercion, whether Source identifies the return statement or the function, and whether finally blocks have completed. Event-versioning is therefore similar to an API and a logical signature at once.

Illustrative event envelope. Concrete field sets should be event-specific rather than universally nullable.

```
type SemanticEvent struct {
  SchemaVersion uint32
  Sequence      uint64
  Kind          EventKind

  ProgramID      ProgramID
  FunctionID     FunctionID
  CallSiteID     CallSiteID
  FrameID        FrameID
  JobID          JobID

  Source         SourceSpan
  Observation     ObservationHandle
  Cause          EventID
}
```

A good schema distinguishes mandatory fields from event-specific variants. Large generic records are convenient initially but weaken static checking, waste bandwidth, and encourage consumers to infer semantics from absent fields. A tagged sum of event-specific records better matches the formal event alphabet and the probe verifier.

5.3 Event tiers and observational power

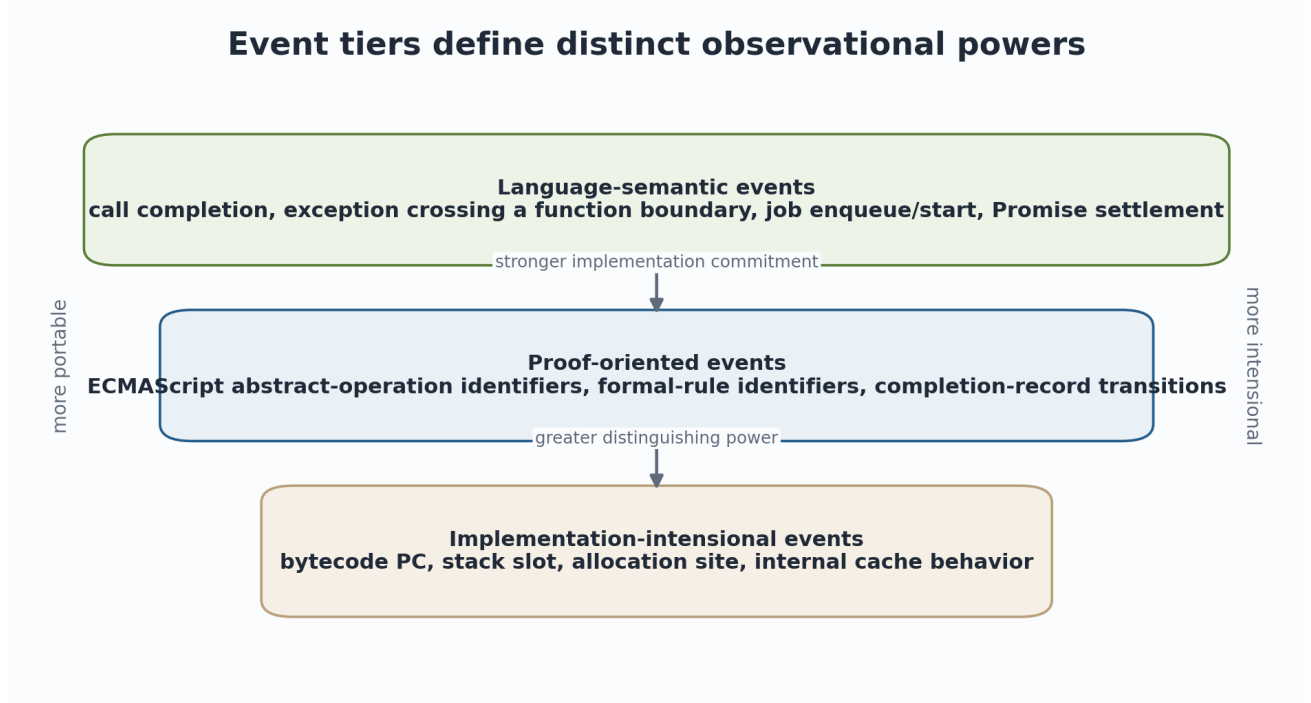


Figure 3. Event tiers range from portable language events to implementation-intensional events.

Language-semantic events aim to be stable across correct implementations of the same language and host model. Proof-oriented events expose the chosen specification decomposition, such as an ECMAScript abstract-operation identifier or a Redex/K rule label. Implementation-intensional events expose goja-specific program counters, stack layouts, allocations, or caches. All three are legitimate, but they support different claims. A performance paper may require implementation events; a source equivalence argument should normally avoid them.

Capabilities should therefore include an event tier. A portable probe may attach only to language-semantic events and use stable value projections. A proof adapter may additionally request rule labels tied to a formal-model version. A VM diagnostics probe may request raw program counters and instruction classes but becomes explicitly tied to a goja commit and compiler configuration.

5.4 Event algebra and closure properties

An event vocabulary is useful when it supports algebraic composition. Event filters should be closed under conjunction and disjunction; trace projections should preserve causal identities; handler products should combine independent analyses; and schema evolution should specify whether older event interpretations remain valid. A probe compiler can exploit these properties by resolving selectors to compact site sets and fusing compatible handlers.

Table 4. Representative semantic event families.

Event family	Typical fields	Common interpretations
Call / construct	callee, receiver, arguments, call site, new target	latency, contracts, call graph, policy
Function lifecycle	function, frame, entry/return/throw, result or exception	pre/post invariants, recursion, summaries
Object effects	base identity, property key, descriptor mode, value projection	shape analysis, taint, effect summaries
Jobs / async	job, Promise, continuation, cause, realm/host queue	causal traces, protocol monitors, scheduling studies
Control flow	branch site, outcome, rule ID	coverage, path profiles, symbolic alignment
Resource	allocation site, size class, operation count	cost semantics, leak detection, resource contracts

6. From VM instructions to semantic macro-events

The event mapping is an abstraction proof problem: sequences of implementation transitions must implement one declared semantic transition.

6.1 Why source rewriting is insufficient

Source or AST rewriting can validate user-facing syntax quickly, but it does not provide complete event coverage. JavaScript calls may arise from getters, setters, proxies, bound functions, native functions, constructors, iterators, Promise reactions, and direct evaluation. Source rewriting also changes source positions, stack traces, closure structure, and possibly optimization behavior. It remains useful as a prototype or an independent differential implementation, not as the final semantic source of truth.

At the other extreme, wrapping every VM instruction yields implementation completeness but forces consumers to reconstruct source semantics from unstable stack effects. A semantic event layer should instead intercept central transitions: the call dispatch path, function prologue completion, normal return, exception unwinding, constructor dispatch, generator suspension/resumption, async runner transitions, and job scheduling. The precise patch points are engineering choices; the event contracts are semantic commitments.

6.2 Macro-step simulation

Let α map a VM state V to a source configuration C , possibly partially. A macro-event implementation is sound when a VM segment $V \rightarrow^* V'$ beginning and ending at event boundaries corresponds to a source transition $\alpha(V) \xrightarrow{[e]} \alpha(V')$. Internal VM steps are hidden. The proof may be a forward simulation, backward simulation, bisimulation, or translation-validation check, depending on the desired guarantees and whether compilation is included (Leroy 2009; Pnueli, Siegel, and Singerman 1998).

$$V \rightarrow^* V' \quad \text{and} \quad \alpha(V)=C \quad \text{imply} \quad C \xrightarrow{[e]} \alpha(V')$$

Schematic event-soundness condition for one VM macro-step.

For a realistic interpreter, a full mechanized refinement proof is a major project. A staged strategy is still rigorous: prove the property for a small abstract machine; encode runtime assertions over event preconditions and postconditions; differential-test events against a specification-oriented interpreter; use semantic-rule coverage; and pin every trace to the implementation and schema version. Translation validation can check per-program metadata and compilation facts without requiring a once-and-for-all proof of the entire compiler.

6.3 Normal return and abrupt completion

Normal return is not merely execution of a return opcode. A return expression may transfer control through one or more finally blocks, and a finally block may replace the pending completion. The semantic return event should therefore occur when the final normal completion crosses the function boundary. Likewise, a throw instruction creates an abrupt completion, but `FunctionThrow` occurs only when unwinding crosses that function. If a catch in the same function handles the exception, no function-exit event has occurred.

This difference creates a concrete test oracle. Construct programs with nested try/catch/finally blocks, returns overridden by finally, exceptions rethrown after mutation, and multi-frame unwinds. The event trace should match the formal completion structure, not the surface location of throw or return statements. These cases are essential for event completeness tests.

6.4 Stable site identity

Attaching by function name or line number alone is not stable. Names may be anonymous or duplicated; lines contain multiple expressions; source maps transform positions; and generated code may have no durable path. Compilation should assign `ProgramID`, `FunctionID`, `CallSiteID`, `BindingID`, and `EffectSiteID` values derived from a content hash plus deterministic local numbering. Human selectors resolve to these IDs before execution. The hot path performs integer or bitset membership checks, not string matching.

7. Product semantics for runtime monitors

A monitor is a second state machine whose product with the program machine yields the monitored execution.

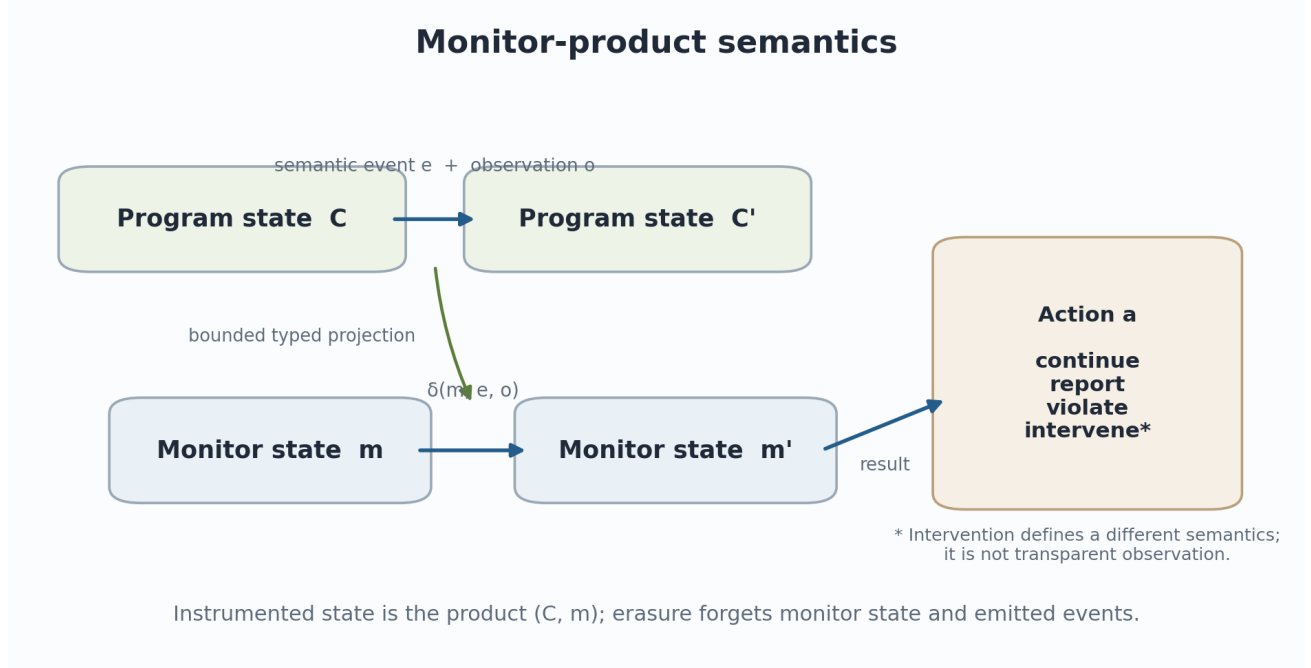


Figure 4. Program and monitor evolve together through an event and bounded observation.

7.1 The basic product construction

Let M be monitor state, E the event alphabet, O the observation domain, and A the action domain. The monitor transition function $\delta : M \times E \times O \rightarrow M \times A$ is deterministic for a fixed probe program, or explicitly nondeterministic if the specification requires it. The monitored configuration is the pair (C, m) . Internal program transitions leave m unchanged; labelled transitions update both components. This construction is standard for automata-based runtime verification and makes stateful invariants mathematically explicit (Leucker and Schallhart 2009; Bartocci and Falcone 2018).

$$\delta : M \times E \times O \rightarrow M \times A$$

A is normally Continue, Report, or Violate; intervention requires a different transition system.

7.2 Safety properties and bad prefixes

Many runtime-enforceable invariants are safety properties: every violation has a finite bad prefix. Examples include a balance becoming negative, a file being used after close, a transaction committing before begin, a Promise continuation resuming twice, or an exception crossing a forbidden boundary. The monitor reaches a bad state as soon as the prefix suffices to demonstrate failure. Schneider formalizes the relationship between security policies and execution monitors; edit automata extend monitors with suppression and insertion capabilities (Schneider 2000; Ligatti, Bauer, and Walker 2005).

The architecture should report the property class. A finite-run monitor can return satisfied, violated, or inconclusive. A liveness property such as every request is eventually answered cannot generally be confirmed from an arbitrary finite prefix. End-of-run assumptions, time bounds, fairness assumptions, or conversion to a bounded response property must be explicit (Alpern and Schneider 1985; Bauer, Leucker, and Schallhart 2010).

7.3 Parametric monitor instances

Events frequently carry data parameters: a transaction ID, object identity, Promise ID, request key, or module instance. Parametric monitoring treats a property as a family of monitor instances indexed by compatible parameter bindings. The global trace is sliced into the events relevant to each binding, and the base monitor runs on each slice. This avoids writing one enormous monitor state machine and provides a semantics for per-object typestate or per-request protocols (Rosu and Chen 2012; Havelund et al. 2018).

Typed maps can implement this efficiently, but map lifetime is part of semantics. The system needs explicit creation, garbage collection, eviction, and finalization policies. An LRU map may be acceptable for telemetry but unsound for proof-mode monitoring because eviction silently forgets obligations. Proof mode should use non-evicting bounded maps, reject execution when capacity is exceeded, or prove a bound on live instances.

7.4 Monitor composition

Independent monitors compose by product: $\delta_1 \times \delta_2$ updates both states on each event. The practical engine can fuse event filtering and share decoded observations, but verdict semantics must define conflicts. If one monitor reports and another violates, violation dominates. If an intervention-capable monitor rewrites an argument, later monitors must know whether they observe the original or rewritten value. This is another reason to keep intervention in a separate capability and event phase.

8. Correctness obligations for the event layer

Probe safety is necessary, but the scientific value of the system depends on stronger obligations connecting events, traces, and source behavior.

8.1 Erasure and transparency

For a purely observational probe, erasure forgets monitor state and event records. A strong theorem states that erasing an instrumented execution yields the same source behavior as the uninstrumented execution. In practice the statement must identify observables. If JavaScript can observe wall-clock duration, allocation pressure, finalization timing, host scheduling, or resource exhaustion, instrumentation can perturb behavior even without modifying language values. A realistic theorem may establish functional transparency under a host abstraction while documenting that quantitative observations are outside the relation.

```
erase(run_instrumented(p, m)) observationally-equivalent-to run(p)
```

The equivalence relation must state which host and quantitative observations are ignored.

Weak bisimulation is often appropriate because probe execution introduces internal administrative steps that should be hidden. Contextual equivalence or trace equivalence may be stronger or weaker depending on the event vocabulary and host model. The document therefore avoids the unqualified claim zero semantic effect; it asks for a theorem relative to a declared observation model (Pitts 1997; Sangiorgi 2011).

8.2 Event soundness, completeness, and uniqueness

Table 5. Principal event-correctness obligations.

Property	Required statement	Typical defect
Soundness / adequacy	Every emitted event corresponds to a declared source or specification transition.	FunctionReturn emitted before finally changes the completion.
Completeness	Every selected transition emits an event when the event family is enabled.	Native or proxy-mediated calls bypass the central hook.
Uniqueness	A transition produces the declared number of events, usually exactly one.	Return is emitted both at opcode execution and at frame pop.
Ordering	Events respect program order and declared causal order.	Async resume appears before the job-start event that caused it.
Projection validity	Each observation field is derived from the promised pre- or post-state.	Arguments are read after the stack has been repurposed.
No silent loss	Proof-mode streams cannot drop, sample, or evict relevant events/state.	Ring-buffer overflow is reported only as a metric.

8.3 Monitor compiler correctness

If a temporal formula, contract, or automaton is compiled to the probe IR, a separate theorem should relate source-property semantics to IR execution. The runtime verifier may establish that the compiled program is well typed and bounded without establishing that it implements the formula. A trustworthy toolchain therefore separates the property compiler, its proof or tests, and the low-level verifier. This separation mirrors proof-carrying code: a small checker validates evidence supplied by a more complex producer (Necula 1997).

8.4 Refinement from goja to JavaScript semantics

The strongest long-term goal is a refinement statement relating goja traces to a formal JavaScript semantics. Because goja implements a practical language subset plus extensions, the relation must account for host functions, native objects, event-loop behavior, unsupported features, and implementation choices. The initial relation can be partial: for executions staying inside a modeled subset and using native modules with declared summaries, normalized goja traces should belong to the trace set of the formal model.

RESEARCH GROUNDING | Evidence hierarchy

Unit tests show local event order for selected cases.

Differential semantic tests show agreement with an independently implemented model on generated programs.

Translation-validation checks establish per-program metadata and compiler facts.

A mechanized simulation establishes the general correspondence theorem for a defined language subset.

9. Exceptions, constructors, generators, async functions, and jobs

Control effects expose where naive enter/return hooks fail and where the event model must represent suspension, unwinding, and host scheduling explicitly.

9.1 Exceptions and finally

An exception has at least three distinct event points: creation of an abrupt completion, handling by a catch or finally mechanism, and propagation across a function or host boundary. Runtime verification often cares about the boundary crossing because that determines which function failed to establish its postcondition. Debugging may also need the original throw site. The schema should expose separate Throw, Catch, FinallyEnter, FunctionThrow, and HostThrow events or a smaller vocabulary with explicit phases.

The unwind algorithm must emit exit events for every discarded frame before its state becomes unavailable. It must not run ordinary JavaScript helpers during unwinding, because reentrancy can mutate the very stacks being inspected. Probe execution should use only the bounded observation VM and should have a defined failure policy if a probe itself reports a violation during an existing abrupt completion.

9.2 Constructors and derived this

Constructor semantics differs from ordinary calls. A base constructor creates or receives an instance; a derived constructor cannot access this before super succeeds; returning an object can replace the constructed instance; returning a primitive from a derived constructor is an error. ConstructBefore, ConstructAfter, and ConstructThrow therefore need constructor identity, new-target identity, argument view, derived/base status, and the final instance. A pre-entry function probe cannot always expose this because it may not yet exist.

9.3 Generators

Invoking a generator function creates a generator object without immediately running the body. The first next call enters the body; yield suspends a continuation; later next, throw, or return calls resume it; completion is distinct from ordinary function invocation return. Treating generator invocation as FunctionEnter would be semantically wrong. The event vocabulary needs GeneratorCreate, GeneratorEnter, Yield, Resume, Complete, and GeneratorThrow, correlated by generator and continuation identities.

9.4 Async functions and Promise jobs

An async function invocation normally returns a Promise before the eventual computation settles. FunctionReturn at invocation time therefore means returned the Promise, not completed the async computation. Await suspends the current async continuation and schedules resumption through Promise machinery. A later job resumes the continuation and may resolve or reject the outer Promise. Runtime contracts often need both levels: a synchronous contract over the returned Promise and an eventual contract over settlement.

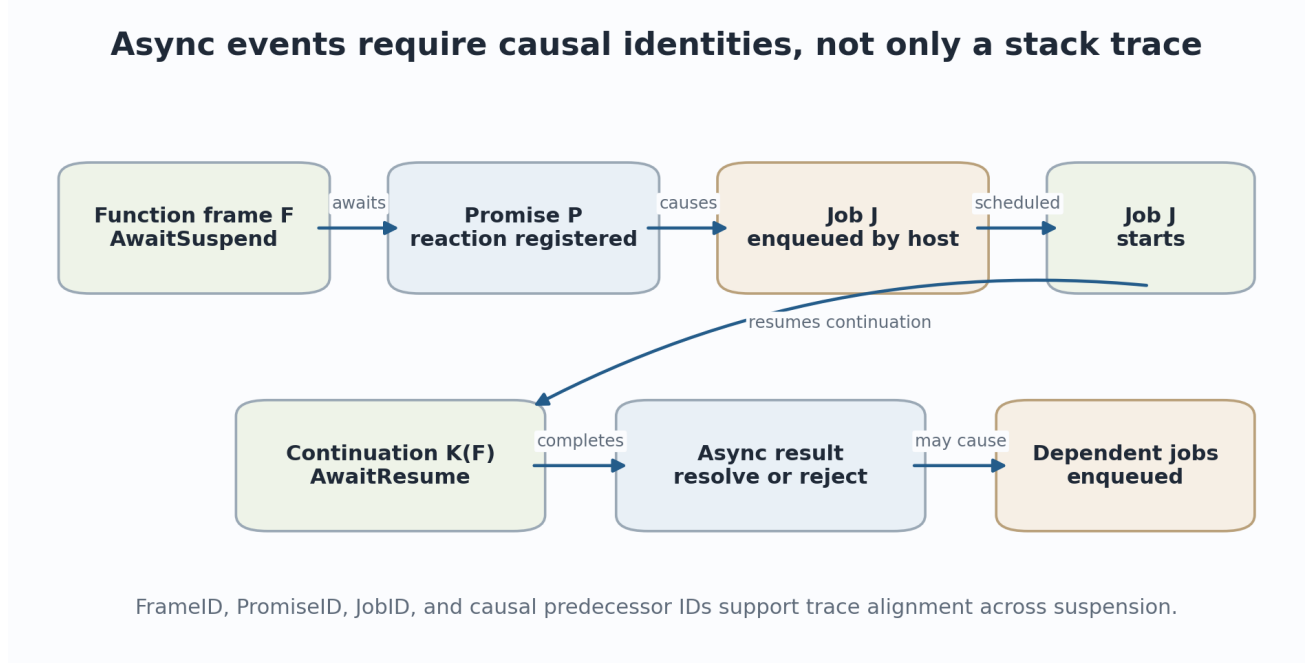


Figure 5. Async traces require frame, Promise, job, continuation, and causal identities.

ECMA-262 defines Jobs as abstract closures scheduled by host operations and states constraints on when they run; actual priority and surrounding event-loop behavior remain host responsibilities (ECMA International 2026). go-go-goja uses an event-loop and runtime-owner abstraction, so a complete semantic trace must include both core ECMAScript job events and host scheduling events. Otherwise two traces may agree on JavaScript function calls while disagreeing on causality and ordering.

9.5 Dynamic code and reflection

Direct eval, Function construction, module loading, and generated code threaten closed-world analysis. The implementation should route every dynamic compilation through the metadata pipeline, assign a content-derived ProgramID, apply the current probe-attachment policy, and record the origin event that caused compilation. Proof modes may instead reject dynamic code or require a trusted loader to provide a preverified artifact. Reflective constructs such as with, proxies, and dynamic property access should be modeled conservatively and marked as precision barriers in static analysis.

10. Observation, enforcement, and intervention

These modes have different operational meanings and should be distinct capabilities, APIs, trace records, and research claims.

10.1 Observation

Observation reads a bounded projection, updates monitor-local state, and emits telemetry. It cannot change arguments, results, exceptions, JavaScript heap state, scheduling, or module availability. Observation is the mode for which an erasure theorem is plausible. Metrics, call graphs, coverage, invariant mining, trace generation, and most proof debugging belong here.

10.2 Enforcement

Enforcement adds a distinguished violation outcome. A monitor recognizes a bad prefix and terminates or quarantines the execution. The monitored semantics therefore accepts a subset of original executions plus explicit violation results. Whether the violation is catchable by JavaScript is a policy choice. For trusted invariants, an uncatchable host error prevents the program from suppressing the check; for ordinary contracts, catchable exceptions and blame objects may be more appropriate.

10.3 Intervention

Intervention can suppress a call, rewrite an argument or return value, synthesize an exception, or modify selected state. Edit automata and aspect systems provide relevant theory, but transparency no longer applies (Ligatti, Bauer, and Walker 2005; Kiczales et al. 1997). Intervention is useful for fault injection, security policy, dynamic repair, and experimentation. The trace must record both the original event and the intervention so that downstream analyses do not mistake modified behavior for evidence about the original program.

Table 6. Semantic distinction among probe modes.

Mode	Allowed actions	Suitable claims
Observe	Read typed projections, maps, metrics, bounded trace emission	Functional transparency relative to declared observables; evidence about the original execution.
Enforce	Observe plus terminate/report a violation	Safety enforcement; execution accepted only while monitor remains non-bad.
Intervene	Rewrite, suppress, insert, or mutate under explicit capabilities	Behavior of an extended or repaired semantics; not evidence of original-program equivalence.

10.4 Phase ordering

Even within one mode, phase ordering matters. CallBefore observes raw arguments at the call site; FunctionEnter observes parameters after function prologue initialization; FunctionReturn observes the callee completion; CallAfter observes the caller-visible result. Native calls may complete synchronously between CallBefore and CallAfter. Proxies or wrappers may create nested call events. The schema should state whether monitors execute before or after each transition and whether observations are immutable snapshots or borrowed live handles.

11. Causality, host semantics, and multi-agent executions

A stack is enough only for synchronous single-agent control; async jobs, host callbacks, workers, and shared memory require explicit causality.

11.1 Happens-before and causal predecessors

Lamport's happens-before relation distinguishes causality from physical time. Event records should carry monotonically ordered Sequence values within one runtime owner and explicit causal predecessor IDs across enqueue/start, Promise/reaction, await/resume, callback registration/invoke, and message send/receive boundaries (Lamport 1978). Wall-clock timestamps are useful for performance but do not define semantic ordering in the presence of suspension or multiple agents.

11.2 Event structures and partial orders

A linear trace is convenient for replay, but concurrency semantics is often better represented as a partial order of events with causality and conflict. Event structures model events together with causal dependency and incompatible alternatives (Winskel 1986). Even if the first goja implementation is single-owner, using causal IDs now avoids baking a stack-only model into the ABI and supports future workers, host concurrency, and multi-runtime analyses.

11.3 Core language versus host model

JavaScript semantics is divided between ECMAScript and the host. Timers, files, network callbacks, process operations, module resolution, and event-loop priorities are host-defined. go-go-goja composes a VM with an event loop, module registry, runtime owner, and native modules. Formal claims should therefore name two models: the core ECMAScript transition system and the host transition system. A HostCall event can be treated as an uninterpreted effect with a contract, a recorded oracle result, or a fully modeled operation.

Native modules require effect summaries if static or proof analyses cross them. A summary may specify argument and result types, heap regions read or written, possible exceptions, scheduling effects, determinism, and resource cost. Without a summary the call is a semantic oracle. The runtime trace can still record its concrete behavior, but a universal proof must assume or verify a contract for the native component.

11.4 Replay and nondeterminism

A replayable trace should record nondeterministic inputs and scheduling choices at the abstraction boundary: time, randomness, I/O results, host callback order, module contents, and external cancellation. Replaying every VM instruction is unnecessary if the semantic events and host oracles are sufficient to reproduce the chosen observable behavior. The artifact should state whether replay is exact, functionally equivalent, or merely trace-checking.

LIMITATION / FAILURE MODE | Host-model discipline

No semantics or proof about async JavaScript is complete without stating the host scheduling assumptions. Core Promise-job ordering does not by itself define timers, I/O callbacks, priorities, cancellation, or interactions among runtimes.

Part III. Denotational and compositional interpretations

How an execution becomes a trace-producing computation and how handlers assign different meanings to the same event stream.

12. Trace denotations and infinite behavior

Denotationally, the event layer enriches the meaning of a program from a final result into an observable behavior that may be finite, infinite, branching, or interactive.

12.1 From state transformers to traces

A simple deterministic state semantics maps an initial store to a result and final store. Instrumentation enriches this with a trace of semantic events. For terminating computations one can write $[[p]] : \text{Sigma} \rightarrow \text{Trace}(\text{E}) \times \text{Outcome} \times \text{Sigma}$. This is already sufficient for many metrics, contracts, and differential tests. However, JavaScript programs may diverge, interact with a host, schedule future jobs, or depend on nondeterministic inputs. A finite-list trace domain does not capture these behaviors adequately.

$$[[p]]_E : \text{Sigma} \rightarrow \text{Trace}(\text{E}) \times \text{Outcome} \times \text{Sigma}$$

A basic trace-enriched denotation for terminating deterministic executions.

A richer semantics uses finite and infinite traces, resumptions, trees, or coinductive processes. Prefix ordering is natural for observations: a longer trace contains more execution evidence than its prefixes. Divergence may produce an infinite sequence of events, an infinite sequence of silent steps, or a finite visible trace followed by silent divergence. These distinctions matter for runtime monitoring and equivalence. A probe that observes only function boundaries cannot distinguish two programs that diverge internally after the same last event.

12.2 Trace domains and partiality

A trace domain should distinguish normal completion, abrupt completion, cancellation, resource failure, and nontermination. It should also separate program events from monitor failures and trace-loss conditions. A proof-mode ring-buffer overflow is not a JavaScript exception and should not be encoded as one; it invalidates the evidence channel. Similarly, an interpreter interrupt may be a host termination rather than an ECMAScript throw.

Table 7. Outcomes that should be separated in a trace semantics.

Outcome	Meaning	Monitor consequence
Normal(v)	JavaScript computation completes normally with value v.	Postconditions and completion properties can be evaluated.
Abrupt(x)	Uncaught JavaScript abrupt completion or declared exception boundary.	Exceptional postconditions and throw policies apply.
Violation(id)	Enforcement monitor recognizes a bad prefix.	Execution belongs to monitored semantics, not the original completed behavior.
Interrupted(r)	Host cancels or interrupts the runtime.	Liveness properties are usually inconclusive unless cancellation is part of the property.
EvidenceLost(r)	Required event or monitor state could not be retained.	Proof or conformance claim is invalid; fail closed in strict mode.
Diverges	Execution does not complete.	Only prefix properties and coinductive observations are available.

12.3 Trace equivalence and normalization

Differential semantics requires a normalization function because two implementations rarely emit identical low-level traces. Normalization may erase implementation events, alpha-rewrite object and frame identities, collapse administrative transitions, and translate source positions through source maps. The comparison relation can then be equality, prefix equivalence, weak bisimulation, or a domain-specific relation that permits reorderings of independent events.

Normalization is part of the trusted methodology. An overly aggressive normalizer can hide real discrepancies; an overly intensional one reports irrelevant implementation differences. The event tiers in Chapter 5 help define explicit comparison profiles: language-level, specification-rule-level, or implementation-level.

13. Monads, algebraic effects, and event handlers

The same semantic events can be interpreted as logging, monitoring, metrics, symbolic constraints, abstract effects, or proof obligations by changing the handler.

13.1 Computational effects and monads

Moggi's computational lambda calculus and monadic semantics separate pure values from computations with effects. A trace-producing interpreter can be represented using a writer-like effect; stateful monitoring adds state; exceptions and nondeterminism add further structure (Moggi 1991). The key architectural lesson is separation: the syntax and control structure of the computation need not hard-code every interpretation of its effects.

For semantic probes, the VM emits operations from an event signature. A handler interprets each operation in a target semantic domain. The runtime handler updates concrete maps; an offline handler appends a trace; a cost handler accumulates weights; a symbolic handler generates formulas; an abstract handler updates lattice elements. This decomposition gives a principled account of the proposed plugin architecture.

13.2 Algebraic effects

Algebraic effect theory treats effects as operations satisfying equations, and handlers provide models of those operations. Plotkin and Power develop algebraic operations and generic effects; Plotkin and Pretnar give a programming-language account of effect handlers (Plotkin and Power 2003; Plotkin and Pretnar 2009). A semantic event such as Call, ReadProperty, Allocate, or EnqueueJob can be regarded as an operation whose continuation represents the remainder of execution.

Not every event is algebraic in a simple sense. Control operators, continuations, observation of identities, and host scheduling can require richer structures. Nevertheless, effect signatures are a useful design discipline: each helper and event has an explicit type, and each interpretation must say what it does with that operation. This discourages hidden access from a monitor into arbitrary interpreter internals.

13.3 Handler products and modular analyses

If two handlers consume the same event alphabet independently, their product computes both results in one pass. A call event can update a counter, a call graph, a temporal monitor, and a cost model simultaneously. The mathematical product corresponds to an implementation that fuses filtering, decodes observations once, and updates several typed states. The composition law also provides a test oracle: the fused result should equal running the handlers separately over the same trace.

$$H1 \times H2 : \text{Event} \rightarrow (M1 \times M2)$$

Product handlers support modular analyses and a fusion correctness test.

13.4 Handler capabilities

Handlers differ in the effects they are allowed to perform. A pure trace fold is deterministic and replayable. A runtime metric handler may use monotonic counters. An enforcement handler can terminate. An intervention handler can modify the continuation or state. Capability typing should make these distinctions explicit, because laws valid for observational handlers may fail for stateful or intervention handlers. For example, handler order is irrelevant for independent commutative counters but observable when one handler rewrites a return value consumed by another.

14. Interaction trees and weak bisimulation

Interaction trees provide a concrete candidate for representing recursive, effectful, and potentially nonterminating JavaScript behavior in a proof assistant.

14.1 Event trees

Interaction trees are coinductive trees built from return nodes, silent steps, and visible events with continuations. They can represent recursive and impure computations while keeping events uninterpreted until a handler assigns meaning. Their equational theory uses weak bisimulation to ignore finite sequences of silent steps, and the Coq library supports executable extraction and compiler-correctness proofs (Xia et al. 2020).

$$\text{ITree } E \ R \ ::= \ \text{Ret } R \mid \text{Tau } (\text{ITree } E \ R) \mid \text{Vis } E \ (X \rightarrow \text{ITree } E \ R)$$

Schematic presentation: a visible event carries a continuation indexed by its result type.

This matches the semantic-probe architecture closely. The source semantics or interpreter abstraction produces an event tree. A trace handler interprets events into lists or streams; a monitor handler threads monitor state; a host handler supplies I/O results; a symbolic handler returns symbolic values. The same event tree can be reasoned about compositionally and executed after handlers are selected.

14.2 Why weak bisimulation matters

Instrumented and uninstrumented executions often differ by administrative steps. Compilers also introduce stack manipulation, environment setup, and helper transitions that have no source-visible counterpart. Weak bisimulation relates processes while abstracting from finite silent behavior. It is therefore a natural candidate for stating that an event-instrumented interpreter preserves source behavior modulo probe-internal steps and selected implementation details (Sangiorgi 2011; Xia et al. 2020).

The event ABI must identify which steps are silent. If bytecode instruction events are enabled, they are no longer silent for that observer. Thus weak equivalence is capability-indexed: a pair of executions may be equivalent to a language-level observer but not to an implementation-level probe. This observation leads directly to the full-abstraction questions in Chapter 16.

14.3 Executable denotations and differential testing

An ITree-based semantics can be extracted to an executable reference model. The goja trace can be normalized to the same event signature and compared against an interpreted ITree. When traces diverge, the system can show the source site, event kind, formal continuation, and concrete VM state projection. This is a denotational counterpart of JSExplain's double-debugger idea and KJS's executable semantics, with the additional benefit that event handlers and equivalence laws can be mechanized (Park, Stefanescu, and Rosu 2015; Chargueraud, Schmitt, and Wood 2018; Xia et al. 2020).

14.4 Limits of the correspondence

Interaction trees do not remove modeling obligations. Object identity, weak references, garbage collection, host scheduling, shared memory, and native modules require suitable event signatures and handlers. An extracted model may be much slower than goja and may represent specification decomposition differently. The value is a shared compositional interface, not automatic proof of equivalence.

15. Metrics, costs, and folds over traces

Metrics become semantically meaningful when their aggregation laws are explicit and when implementation timing is separated from abstract cost.

15.1 Monoids and folds

Many online metrics are folds over an event trace. A monoid supplies an identity element and associative combination. Counts use natural-number addition; sets of covered rules use union; maximum stack depth uses max; call multisets use

pointwise addition. If a metric is a monoid homomorphism, results can be computed online, recomputed offline, partitioned across trace chunks, and merged without dependence on chunking.

$$\text{fold}(\text{epsilon}) = \text{identity} \qquad \text{fold}(t1 \mathrel{++} t2) = \text{fold}(t1) \text{ combine } \text{fold}(t2)$$

Associative aggregation is the algebraic basis for streaming and parallel trace analysis.

This is not only elegance. It lets the runtime use compact aggregations while an independent offline implementation checks the same result from a sampled or complete trace. It also identifies metrics that are not safely mergeable. Percentiles require sketches with stated error bounds; wall-clock inclusive time requires careful nesting and causal accounting; eviction or sampling changes the algebra and must be represented in metadata.

15.2 Semantic cost versus measured time

A cost semantics assigns abstract weights to semantic transitions. For example, one can count property operations, allocations, host crossings, Promise jobs, or selected abstract operations. The result is stable under many machine-level fluctuations and can support asymptotic or resource-contract reasoning. Measured elapsed time, by contrast, includes interpreter implementation, Go runtime scheduling, garbage collection, blocking host calls, and probe overhead.

Both are useful, but they answer different questions. A semantic cost model asks how much abstract work an execution performs. A performance profile asks how long the current implementation takes. Calibration can regress measured cost against semantic counts, but the document should not present one as the other. Goja's existing profiler already distinguishes execution time from CPU time and samples at interpreter instruction boundaries; semantic probes can complement it with operation-specific counts and causal host timings (Dop251 2026).

15.3 Weighted automata and resource monitors

Temporal monitors can be enriched with weights. A request protocol may accumulate bytes, allocations, or abstract gas while moving through states. Semirings and weighted automata provide a general language for combining path alternatives and sequential costs. A practical first version can use typed counters and histograms while retaining a formal interpretation as folds over labelled transitions.

15.4 Perturbation and experimental design

Instrumentation changes the system being measured. A no-op event branch changes instruction-cache behavior; observation allocation changes garbage collection; ring-buffer writes add synchronization or memory traffic; detailed timing changes scheduler interaction. Performance studies therefore need a tiered benchmark: uninstrumented, event machinery compiled but disabled, event family enabled with no attached probe, no-op probe, counter-only probe, contract probe, trace-emitting probe, and full offline export. Results should be reported per event family and workload class.

16. Observational equivalence and full abstraction

Adding probes creates new observers. The event capability determines which programs and implementations can be distinguished.

16.1 Contextual equivalence

Two programs are contextually equivalent when no admissible program context can distinguish them according to a chosen observation, such as termination or returned values. Operational theories use bisimulation, logical relations, or contextual reasoning to characterize such equivalences (Milner 1977; Plotkin 1977; Pitts 1997). A probe layer expands the class of observers beyond ordinary JavaScript contexts.

Consider two functions that return the same value but one calls a helper. A result-only JavaScript context may not distinguish them; a function-entry probe does. Two implementations may agree on source semantics but differ in allocation sites or bytecode instruction counts; an implementation probe distinguishes them. Therefore equivalence must be indexed by the event capability and observation helpers.

$$p \sim_{\text{JS}} q \quad \text{does not necessarily imply} \quad p \sim_{\text{ProbeCapability}} q$$

A stronger observer induces a finer equivalence relation.

16.2 Full abstraction

A semantic model is fully abstract when denotational equality coincides with contextual equivalence: the model identifies exactly the programs that no allowed context can distinguish. For semantic probes, a research question is capability-indexed full abstraction. Given an event alphabet and helper set, can one characterize the contextual observations available to probes, and does a trace or interaction-tree model distinguish exactly those behaviors? Milner's and Plotkin's classic work supplies the conceptual standard, even though JavaScript and host effects make the problem substantially harder (Milner 1977; Plotkin 1977).

16.3 Representation independence and opacity

Opaque value handles protect both safety and abstraction. If a probe can read stack indexes, object layout, pointer addresses, or hidden slots, implementation refactoring changes program observations and destroys representation independence. Language-semantic helpers should expose only operations justified by the selected observation model. Implementation-intensional capabilities may expose more, but the resulting probes must be version-pinned and excluded from source-level equivalence claims.

16.4 Event minimization as a design problem

The ideal event ABI is not maximal. Every event increases overhead, trusted code, versioning burden, and observational power. The design problem is to find a sufficient basis from which common analyses can be derived without revealing unstable details. Calls, completions, object effects, jobs, and host boundaries may form such a basis; instruction and allocation events can remain optional tiers. Formal distinguishability results can guide this minimization rather than relying only on intuition.

RESEARCH GROUNDING | Research opportunity

Define a family of probe capabilities and characterize the induced observational equivalences. This turns event-schema design into a full-abstraction and representation-independence problem rather than an ad hoc logging decision.

Part IV. Verified probe execution

A safe extension architecture for running analyses inside the interpreter without exposing raw VM authority.

17. A restricted probe IR and verifier

The probe language should be expressive enough for monitors and metrics, but deliberately weaker than JavaScript.

17.1 Why probes should not be ordinary JavaScript

Calling JavaScript at every semantic event would re-enter the interpreter, allocate objects, invoke getters or proxies, throw exceptions, schedule jobs, access host modules, and execute without a useful static bound. The monitor would become part of the program's dynamic semantics and could recursively trigger itself. Such callbacks are appropriate for a debugging mode, not for a safe, low-overhead, proof-oriented substrate.

A dedicated IR allows the system to control instruction count, stack depth, memory retention, helper effects, branch structure, event emission, and map access. It also makes the probe portable across source syntaxes: users can write a JavaScript-like DSL, a declarative automaton, a contract language, or generated code, all compiling to the same small checked representation.

17.2 Core types and values

A first IR needs primitive integers, booleans, bounded byte strings or hashes, event and site IDs, opaque value handles, map references, and fixed-size tuples. It should avoid general heap allocation, recursive data, closures, and unbounded strings. Value handles are affine or borrowed: they are valid only during the current event unless converted to a stable primitive, identity token, or bounded snapshot.

Illustrative probe types and instructions. The exact set should be smaller in the first implementation.

```
type Ty =
  I64 | U64 | Bool | SiteID | FunctionID | FrameID |
  ValueHandle<EventLifetime> |
  MapRef<KeyTy, ValueTy> |
  Bytes<N>

instruction examples:
  const_i64, add_i64, cmp_lt, branch
  arg_count, arg_handle, result_handle
  value_typeof, value_i64, object_id
  own_data_property
  map_lookup, map_update, counter_add
  emit_bounded, violation
```

17.3 Verifier obligations

The verifier should establish register initialization, type consistency across control-flow joins, stack bounds, map key and value schemas, helper signatures, event-phase validity, handle lifetime, maximum instruction count, maximum emitted bytes, maximum map growth, and capability compliance. Version 1 can reject loops and recursion. Later versions may admit statically bounded loops, but verifier complexity and worst-case path exploration should be measured, as eBPF experience shows that boundedness alone does not make verification cheap (Linux Kernel Documentation 2026).

Table 8. Representative probe-verifier checks.

Verifier domain	Example rule	Failure prevented
Register types	A value-handle register cannot be consumed by integer addition.	Memory confusion and undefined helper inputs.
Control-flow joins	All incoming paths agree on register initialization and compatible types.	Path-dependent use of uninitialized state.

Verifier domain	Example rule	Failure prevented
Event phase	result_handle is valid only on return/after events.	Reading a stack location before the result exists.
Lifetimes	Borrowed handles cannot be stored in persistent maps.	Retention of VM objects and use-after-event.
Resources	Instruction, helper, emission, and state budgets have static or checked bounds.	Interpreter starvation and unbounded memory use.
Capabilities	Effectful helpers require an intervention or debugging capability.	Accidental mutation or reentrant JavaScript execution.

17.4 Abstract interpretation inside the verifier

A practical verifier is itself an abstract interpreter. It propagates abstract register types, value ranges, handle provenance, and resource counters over the probe control-flow graph until a fixpoint or acyclic completion. The verifier need not prove the JavaScript property; it proves a safety policy about the probe machine. This follows the broader principle that static verification can be a small abstract semantics for extension code (Cousot and Cousot 1977; Cousot 2021).

17.5 Proof-carrying variants

A later design could allow a producer to supply a proof certificate for richer probe code. The runtime checks the certificate using a smaller validator, following proof-carrying code. Typed assembly language suggests another route: encode safety invariants in the IR type system so that type checking replaces more complex analysis (Necula 1997; Morrisett et al. 1999). The engineering trade-off is between a simple restrictive verifier and a richer certificate format with a more complex toolchain.

18. Opaque handles and effect-controlled state projection

The state interface is where semantic precision, non-interference, memory safety, and performance meet.

18.1 Borrowed observations

Probes should receive event-specific context objects containing opaque handles to this, arguments, results, exceptions, properties, or objects. A handle is not a Go pointer and does not expose arbitrary JavaScript operations. It names a value for the duration of the event and can be passed only to approved helpers. The verifier tracks the handle's lifetime and provenance.

Persistent monitor state should contain copied primitives, hashes, stable runtime-local identities, or bounded snapshots. Storing a raw goja value in a map would retain the object graph, blur runtime ownership, and make monitor lifetime affect garbage collection. Stable object IDs can be weakly associated with objects, but their lifecycle and reuse policy must be specified.

18.2 Pure and effectful observation

A JavaScript property read is not necessarily a pure observation. It can invoke a getter, trigger a proxy trap, allocate, throw, mutate state, or call host code. Contract and proof modes therefore need helpers with restricted semantics, such as reading an existing own data property of an ordinary object without conversion. If the object is a proxy, accessor, dynamic host object, or unsupported exotic object, the helper returns Unknown or a typed failure rather than executing JavaScript.

Table 9. State helpers classified by semantic effect.

Helper class	Semantics	Allowed modes
Pure primitive projection	Read type tag or copy exact primitive value when representation is supported.	Observe, enforce, intervene
Safe structural observation	Read own data descriptor, stable object ID, bounded shape summary; no user code.	Observe, enforce, intervene
Snapshot	Copy bounded depth/bytes under explicit cycle and exotic-object rules.	Observe with budget; proof use requires a model of truncation.

Helper class	Semantics	Allowed modes
JavaScript-semantic read	May invoke getters, proxies, coercions, exceptions, or host code.	Debug/intervention only; not transparent.
Mutation	Replace argument/result or write selected state.	Intervention only.

18.3 Partiality and unknown values

Safe helpers are necessarily partial. A value may not be an integer; an object may be exotic; a property may be absent or accessor-based; a string may exceed the copy budget. The IR should represent `Option`, `Result`, or a small abstract value domain rather than panic or coerce through JavaScript semantics. Monitor source languages can offer pattern matching and explicit policies for `Unknown`. Treating unknown as false is unsound for enforcement; treating it as violation may be safe but overly restrictive.

18.4 Confidentiality and least authority

Instrumentation can expose secrets that ordinary JavaScript modules cannot access: hidden host objects, closure-local values, authentication tokens, or cross-module state. Attach permission and observation capability therefore require a security model. A tenant-level probe may inspect only its modules and redacted values; an operator probe may inspect identities and metrics but not strings; a trusted proof probe may access selected lexical bindings. The event ABI should make redaction and capability checks part of field production, not an after-the-fact exporter filter.

19. Maps, frame-local storage, and parametric monitoring

Persistent typed state turns event callbacks into temporal analyses, but map semantics must be explicit enough for proof and replay.

19.1 Map families

Useful map kinds include fixed arrays, bounded hashes, LRU hashes, monotonic counters, histograms, ring buffers, runtime-local state, function-local state, frame-local state, and finite-state monitor maps. Each map has a key schema, value schema, capacity, eviction policy, concurrency policy, and snapshot/export semantics. As in eBPF, maps separate long-lived state from individual probe invocations; unlike eBPF, they may also encode semantic lifetimes such as a call frame or async continuation.

19.2 Frame-local storage

Entry-to-return contracts need storage automatically associated with a dynamic frame. A monitor may record the entry balance, start time, argument hash, or abstract state and retrieve it at normal or exceptional exit. Implementing this as a general hash keyed by `FrameID` adds overhead and risks leaks. A VM-integrated frame-local map can allocate a fixed monitor slot when a probed frame is created and release it on return, unwind, generator completion, cancellation, or runtime shutdown.

Frame-local state must follow suspension. An async or generator continuation may leave the active stack while remaining semantically live. The slot therefore belongs to the semantic continuation rather than only the physical VM stack. The same design supports proof invariants that relate entry state to eventual settlement.

19.3 Map soundness modes

Telemetry and proof use different failure semantics. A telemetry histogram can drop samples and record a lost count. A proof-mode typestate map cannot silently evict an object whose protocol is incomplete. The map declaration should specify `BestEffort`, `FailClosed`, or `ProvenBound`. `BestEffort` reports loss metadata; `FailClosed` terminates or invalidates the run on capacity failure; `ProvenBound` requires the attachment tool to establish or assume a bound on live keys.

19.4 Trace slicing and garbage collection

Parametric monitoring creates monitor instances on demand. An instance can be reclaimed when the property reaches a terminal state, the parameter object is proven unreachable, or an explicit lifecycle event occurs. Weak identity maps can help

but introduce garbage-collection nondeterminism and proof complications. For reproducible monitoring, explicit lifecycle events are preferable; weak reclamation should be recorded as a semantic event or excluded from claims that depend on instance persistence.

20. Safety, resource bounds, and the trusted computing base

The trust boundary must identify what is proved, what is checked, what is assumed, and what can invalidate an otherwise safe monitor.

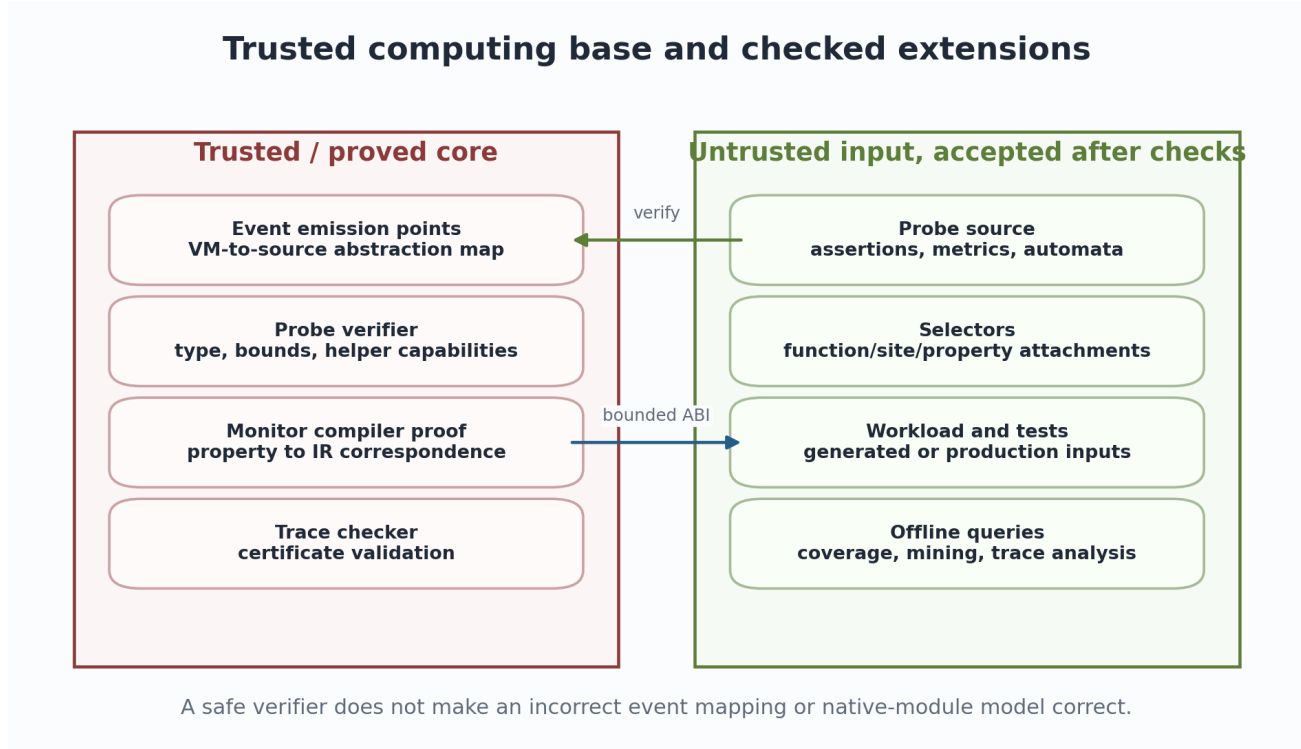


Figure 6. Trusted core versus extension inputs accepted after verification.

20.1 Trusted components

The trusted computing base includes the event emission points, the code that constructs observations, the value-handle implementation, the probe verifier, helper implementations, map implementation, enforcement transition, and any trace certificate checker. If a property compiler is not verified, its output can still be checked by differential tests or a certificate. Native module summaries and host adapters are assumptions unless independently verified.

The Go compiler, runtime, operating system, and hardware are also part of the practical trusted stack unless the project adopts a stronger verified-systems foundation. The document uses trusted computing base in a scoped sense: the components specific to semantic probing whose defects can falsify the event or monitor claim.

20.2 Resource accounting

Every probe invocation should have an instruction budget, helper-cost budget, emitted-byte budget, and possibly a map-operation budget. Static verification can bound acyclic code. Helpers with data-dependent cost, such as bounded snapshot traversal, need cost functions checked against declared limits. The runtime must define what happens when a budget is exceeded: skip and count, disable the probe, report EvidenceLost, or terminate in strict mode.

Aggregate resource use also matters. A safe individual probe can create too many map instances or emit too much data at a high-frequency event. Admission control should compute a worst-case or configured event-rate budget, enforce per-runtime quotas, and separate operator telemetry from proof-mode runs. These controls are analogous to safe extension systems but require semantic event-rate models.

20.3 Reentrancy and failure isolation

Probe execution should not call back into the instrumented JavaScript runtime. Helpers should be written in Go against stable internal accessors and should not allocate JavaScript objects or invoke conversion hooks. Probe panics or verifier bugs must be isolated from the VM where possible; the runtime owner should recover them and produce a probe failure outcome. A proof-mode run should fail closed rather than continue with the monitor disabled.

20.4 Schema and implementation versioning

A probe artifact should declare its ABI version, required event families, helper capabilities, map schemas, source or metadata selectors, and compatibility range. A trace should include the goja commit, go-go-goja commit, compiler options, source hashes, event-schema version, probe hash, verifier version, and native-module manifest. Versioning is a semantic requirement because changing the point at which Return fires or the meaning of a property observation changes monitor results even if the binary layout remains compatible.

LIMITATION / FAILURE MODE | Primary trust lesson

The verifier proves properties of probe execution. It cannot repair an unsound event definition, an incomplete hook path, a misleading native-module summary, or an exporter that silently drops proof-relevant data.

Part V. Static analysis, runtime verification, and proof

How one property artifact can be executed concretely, approximated statically, explored symbolically, and translated into logic.

21. One probe language with four interpretations

The most valuable unification is not a universal solver; it is a shared typed property IR with several interpreters and explicit relations among them.

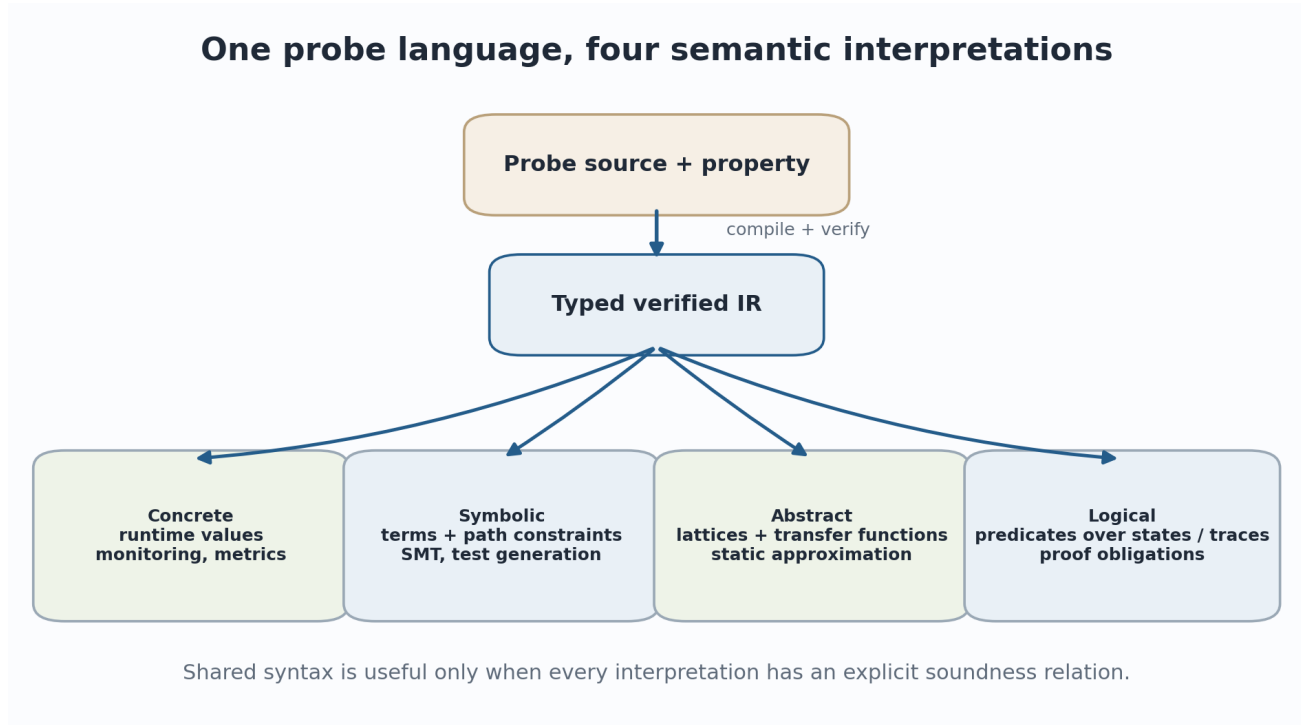


Figure 7. A shared probe IR interpreted concretely, symbolically, abstractly, and logically.

21.1 Concrete interpretation

The concrete interpreter runs inside goja's owner context on actual event records. Primitive projections return actual integers, booleans, strings under bounds, object identities, or Unknown. Map operations update concrete state. Assert produces Continue or Violation. This is the interpretation checked by the probe verifier and benchmarked for runtime overhead.

21.2 Symbolic interpretation

The symbolic interpreter replaces concrete values with terms and path conditions. `arg(0)` may become the symbolic variable `a0`; `own_i64(this,"balance")` becomes a symbolic heap read subject to object-model constraints; a branch adds a path predicate; assert adds an obligation or finds a satisfying counterexample. The interpreter can be embedded in a symbolic executor for the JavaScript subset or used locally over recorded path conditions.

21.3 Abstract interpretation

The abstract interpreter maps values into a chosen abstract domain: type sets, intervals, signs, string abstractions, object shapes, points-to sets, taint labels, or automaton states. Operations are lifted soundly. An assertion is statically discharged when every abstract state implies it; refuted when every state violates it; otherwise it becomes residual. Different domains can interpret the same IR while trading precision for cost (Cousot and Cousot 1977; Cousot 2021).

21.4 Logical interpretation

The logical interpreter translates probe operations into predicates over source or trace semantics. A frame-local assignment becomes a logical variable associated with the entry state. A return assertion becomes a relation between entry and exit states. A temporal monitor becomes an inductive automaton over events. Rocq or Lean can then prove that the monitor compiler preserves property meaning or that a JavaScript program never reaches the bad state under assumptions.

21.5 Soundness relations, not implementation reuse alone

Shared syntax does not guarantee semantic agreement. Each interpretation needs a relation to the concrete meaning. An abstract interpreter should over-approximate concrete outcomes; a symbolic interpreter should cover or explicitly bound paths; a logical translation should preserve the intended assertion; the runtime compiler should implement the IR. Differential tests among interpreters are valuable but do not replace proofs. The architecture should record which obligations are established, assumed, tested, or open.

Table 10. Four interpretations and their distinct correctness obligations.

Interpretation	Domain	Result	Required relation
Concrete	Actual typed observations and map state	Metric, event, or violation	Implements IR operational semantics.
Symbolic	Terms, symbolic heap, path constraints	Models, counterexamples, path formulas	Every reported model corresponds to a feasible concrete execution, or approximation is stated.
Abstract	Lattice elements and transfer functions	Proved, refuted, unknown, residual	Concrete states are contained in abstract states; transfers are sound.
Logical	Predicates, inductive traces, theorem statements	Proof obligation or checked theorem	Translation preserves property semantics under stated assumptions.

22. Abstract interpretation and abstract machines

Abstract interpretation supplies the main theory for turning concrete semantic observations into sound static approximations and for implementing the probe verifier itself.

22.1 Concrete and abstract domains

Abstract interpretation relates a concrete semantic domain to an abstract domain through abstraction and concretization functions, often organized as a Galois connection. Concrete execution computes exact but generally undecidable sets of states; abstract execution computes an over-approximation in a tractable lattice. Fixpoint iteration, widening, and narrowing handle loops and recursion (Cousot and Cousot 1977; Nielson, Nielson, and Hankin 1999; Cousot 2021).

$$\alpha : \text{Concrete} \rightarrow \text{Abstract}$$

$$\gamma : \text{Abstract} \rightarrow \mathcal{P}(\text{Concrete})$$

Soundness requires each concrete state represented by an abstract element to remain represented after abstract transfer.

22.2 Abstracting the JavaScript machine

A JavaScript abstract interpreter must approximate environments, stores, continuations, objects, property keys, prototypes, exceptions, and jobs. Dynamic property names, coercions, eval, proxies, and host functions are major precision challenges. The semantic event ABI can help modularize the analysis: events expose stable effect categories and site IDs, while summaries describe native modules. The event layer does not make JavaScript easy to analyze, but it provides common program points and runtime evidence for evaluating the abstraction.

22.3 Abstracting abstract machines

Abstracting Abstract Machines derives static analyses by bounding components of a concrete abstract machine, especially addresses and stores, then computing reachable abstract states (Van Horn and Might 2010). This methodology aligns well with the proposed operational semantics: begin with the same call, return, exception, and job machine used to define events; abstract its state; and derive an analyzer whose semantic sites share IDs with the runtime.

The benefit is trace alignment. When an abstract state at `FunctionReturn` is imprecise, the workbench can compare it with concrete values observed at the same `FunctionID` and `FrameID` across test runs. This does not justify narrowing the abstract state unsoundly, but it identifies where allocation strategies, join points, widening, or host summaries lose precision.

22.4 Static discharge and residual obligations

Suppose a postcondition requires `balance >= 0`. An interval analysis may prove the property for all reachable returns, refute it, or produce the abstract interval `[-10,+infinity]`, which is inconclusive. The compiler records a proof artifact for discharged sites and emits a runtime monitor only at unresolved sites. The runtime trace can then report whether the residual obligation was actually exercised and whether a counterexample occurred. This is residual verification, not unsound profile-guided proof.

22.5 Invariant mining as hypothesis generation

Dynamic invariant detection systems such as Daikon infer candidate relations from observed executions: ranges, equalities, linear relationships, sequence properties, and implications. The results are likely invariants relative to the test data, not proofs (Ernst et al. 2001, 2007). Semantic probes improve data quality by collecting values at precisely defined entry, return, throw, job, and property-effect events rather than at manually inserted program points.

Candidates can be fed into a static verifier, as explored by work on static verification of dynamically detected invariants (Nimmer and Ernst 2001). The workbench should preserve provenance: which runs supported the candidate, which values were unknown or truncated, which semantic sites were sampled, and whether the static checker discharged the candidate universally.

23. Symbolic execution, concolic testing, and refinement

Symbolic execution turns semantic events and probe assertions into path constraints, while concrete traces guide exploration and counterexample minimization.

23.1 Symbolic execution

Symbolic execution runs a program over symbolic inputs and accumulates constraints along each branch. Solvers produce concrete assignments that exercise paths or violate assertions (King 1976). For JavaScript, the symbolic state includes dynamic types, object properties, prototypes, exceptions, and host effects. A full symbolic engine is difficult; a semantic-probe IR still provides local assertion semantics and a common representation of obligations.

23.2 Concolic execution

Concolic execution combines concrete execution with symbolic tracking. A goja run supplies the concrete path and semantic events; a symbolic companion records predicates at branch or effect sites; negating one predicate produces a new input candidate. Stable SiteIDs make the concrete and symbolic traces align. This can generate tests specifically targeting unresolved contracts, exceptional completions, constructor cases, or async protocol transitions.

23.3 Counterexample-guided abstraction refinement

CEGAR begins with an abstraction, checks a property, analyzes a counterexample, and refines the abstraction when the counterexample is spurious (Clarke et al. 2000). Semantic traces can serve as concrete witnesses during this loop. If the abstract analyzer reports a possible `FunctionThrow` path but no satisfying concrete execution exists, the workbench identifies the branch, property, or host summary responsible for the spurious path and proposes a refinement predicate.

```
abstract model -> candidate counterexample -> feasibility check -> refine or report real
violation
```

Semantic site IDs connect all stages to the same source and VM transitions.

23.4 Differential semantics and test generation

QuickCheck-style property-based generation and Redex random testing show the value of executable models for finding counterexamples (Claessen and Hughes 2000; Felleisen, Findler, and Flatt 2009). The semantic probe workbench can generate JavaScript programs, run them in goja and a reference semantics, normalize both event traces, and minimize the first divergence. Coverage should be measured over formal rules and event cases, not only source branches.

The Run Your Research study found errors while mechanizing published semantics, supporting the practical thesis that lightweight executable formalization pays off during ordinary research rather than only after a mature proof effort (Klein et al. 2012). Semantic probes extend that methodology to a production-style interpreter and its host integration.

23.5 Trace-guided proof debugging

When an inductive proof fails, the missing invariant is often difficult to diagnose. A researcher can compile the proposed induction invariant into event assertions and search concrete executions for the first violation. A violation is not a proof that the theorem is false if the event mapping or abstraction differs from the proof model, but it provides a focused configuration and path. The counterexample can become a proof-assistant test case or reveal that the induction hypothesis is too weak.

24. Contracts, hybrid checking, and residual verification

Contracts give a direct user-facing syntax for semantic probes, while hybrid verification determines which checks remain at runtime.

24.1 Preconditions, postconditions, and blame

A function contract relates calls and completions. Preconditions constrain the caller-provided receiver and arguments; postconditions constrain the result and state relation; exceptional postconditions constrain thrown values; invariants may hold at both entry and exit. Higher-order contract work emphasizes blame: when a boundary contract fails, the system should identify whether the provider or client violated its obligation (Findler and Felleisen 2002; Greenberg, Pierce, and Weirich 2010).

A VM-level contract mechanism can observe all invocation paths, including Go-to-JavaScript calls and native functions, but must preserve the distinction between call-site and function-entry events. Blame labels and module identities should be compiler metadata, not inferred from stack strings at failure time.

24.2 JavaScript contract execution

TreatJS demonstrates higher-order contracts for JavaScript and highlights the need for contract execution to avoid interfering with the program (Keil and Thiemann 2015). The proposed restricted probe VM strengthens this isolation: contract predicates cannot execute arbitrary JavaScript, invoke getters, mutate objects, or run without a bound unless granted an explicit debugging capability. This restriction reduces expressiveness but improves predictability and makes verification tractable.

24.3 Hybrid checking

Hybrid type checking runs static checks where possible and inserts dynamic checks where proof is unavailable (Flanagan 2006). Soft contract verification similarly tries to verify behavioral contracts statically and retain residual runtime checks (Nguyen, Tobin-Hochstadt, and Van Horn 2014). A semantic-probe compiler can implement the same architecture: each contract generates obligations indexed by semantic sites; the static analyzer discharges a subset; the remaining obligations compile to probes.

```
Obligations = Proved_static union Residual_runtime union Rejected_or_unknown
```

The artifact records the disposition and assumptions of every obligation.

24.4 Residual monitor manifests

The build should emit a manifest binding source hashes, compiler options, semantic metadata, discharged obligations, residual probe hashes, native-module assumptions, and proof artifacts. At runtime the manager verifies that the loaded program metadata matches the manifest before attaching monitors. A successful run then supplies evidence only for the residual obligations and only under the manifest assumptions.

24.5 Contracts versus universal proof

A runtime contract checks actual calls. A static proof establishes a property of all modeled calls. A contract that never fires because the function was not exercised provides no evidence about the function. The workbench should report obligation coverage: attached, reachable, exercised, satisfied, violated, or not observed. This prevents a green dashboard from being mistaken for universal verification.

25. Temporal properties and hyperproperties

Single-trace temporal monitoring covers protocols and response properties; relational security and equivalence require multiple traces or static proof.

25.1 Temporal logic over finite traces

Pnueli introduced temporal logic for reasoning about program executions, and automata-theoretic verification connects temporal formulas with automata over traces (Pnueli 1977; Vardi and Wolper 1986). Runtime verification sees finite prefixes, so semantics must distinguish permanent truth, permanent falsehood, and inconclusive status. LTL on finite traces and related logics provide direct finite-run semantics (De Giacomo and Vardi 2013).

A probe property might state: after transaction begin, commit or rollback must occur before the transaction object is released; after AwaitSuspend, exactly one matching resume or cancellation must occur; a module may call exec only after a capability grant event. Such properties compile naturally to deterministic monitors when the relevant parameter domains and event set are bounded.

25.2 Typestate and protocols

Typestate associates permitted operations with an object's abstract state (Strom and Yemini 1986). Semantic probes can enforce typestate dynamically for host resources, iterators, transactions, requests, or Promise-like objects. Static analysis can approximate the same automata. The shared monitor IR makes it possible to compare static typestate results with actual protocol traces and to leave residual checks where aliasing or dynamic dispatch defeats the static analysis.

25.3 Hyperproperties

Hyperproperties are sets of sets of traces; they express requirements relating multiple executions, including noninterference, observational determinism, and some equivalence properties (Clarkson and Schneider 2010). A single-run monitor cannot establish general noninterference because the property compares outputs across secret variations. HyperLTL and monitoring algorithms for hyperproperties provide a formal basis for multi-run trace analysis (Clarkson et al. 2014; Finkbeiner et al. 2017).

The workbench should include a hyper-monitor layer that groups traces by an input relation, alpha-renames identities, aligns events, and compares selected observations. Examples include differential testing of two interpreter versions, checking determinism under repeated recorded host inputs, studying constant-operation traces for selected secrets, and validating an optimization against an unoptimized build. These are stronger claims than ordinary runtime monitoring and require a carefully defined trace relation.

25.4 Sampling and temporal soundness

Sampling is usually incompatible with exact temporal monitoring. Missing one begin or close event changes the protocol trace. The event manager should classify analyses as sample-safe or exact. Counts and statistical latency profiles may tolerate sampling; safety monitors, trace certificates, semantic coverage, and hyperproperty alignment generally cannot. When events are sampled, the trace schema should record the sampling process so that statistical estimators can be interpreted correctly.

26. Proof-relevant traces and independently checked evidence

A proof-relevant trace records not only what happened but enough bounded witness information to check why a declared transition was valid.

26.1 From logs to certificates

Ordinary logs are intended for humans and often omit state required to validate a claim. A proof-relevant event includes a rule or operation identifier, source and semantic site IDs, causal predecessors, selected pre-state and post-state projections, and a witness appropriate to the transition. For a property write, the witness may include the property key category and descriptor mode; for a return, the completion kind and frame relation; for a job start, the enqueue event and queue class.

The checker should be smaller and simpler than the producer. It can validate local transition schemas, causal consistency, monitor-state updates, map capacity facts, and trace digests. This resembles proof-carrying code and translation validation: complex producers generate evidence, while a small checker decides whether to accept it (Necula 1997; Pnueli, Siegel, and Singerman 1998).

26.2 Local versus global checking

Local checking verifies each event against adjacent or referenced records. Global checking validates nesting, uniqueness, causal acyclicity, protocol completeness, and source metadata. Some JavaScript transitions require more state than can be retained in every event. The design can use periodic state commitments or hashes, with selective expanded witnesses for audited events. Cryptographic integrity protects against accidental or malicious modification but does not establish semantic correctness of the producer.

26.3 Certificates for one run are not universal proofs

An independently checked trace certifies a particular execution under a particular model and manifest. It can show that the monitor updated correctly and that no recorded bad state occurred. It does not establish that all inputs or schedules satisfy the property. Universal proof requires induction, model checking, abstract interpretation, symbolic coverage, or another argument over the entire semantics.

26.4 Reproducible artifact bundle

Table 11. Components of a reproducible semantic-probe artifact.

Artifact	Purpose
Source and dependency hashes	Identify the exact JavaScript, goja, go-go-goja, native modules, and formal model.
Semantic metadata	Relate source spans, functions, sites, bindings, rules, and VM program locations.
Probe source and verified IR	Allow independent inspection of the property and executed monitor.
Manifest and assumptions	Declare capabilities, discharged/residual obligations, host contracts, and limits.
Trace or aggregate certificate	Record events, causal links, monitor transitions, loss status, and digests.
Independent checker and queries	Recompute verdicts, coverage, metrics, and selected transition checks.

Part VI. JavaScript, goja, and the academic workbench

Concrete implications for ECMAScript, goja internals, go-go-goja integration, and everyday research practice.

27. JavaScript semantics and executable specifications

JavaScript is a demanding test case because specification devices, dynamic language features, host effects, and implementation structures do not align trivially.

27.1 ECMAScript as the reference specification

ECMA-262 defines algorithms, completion records, environment records, execution contexts, Jobs, object internal methods, and host hooks. It explicitly states that execution contexts are specification devices and need not correspond to implementation artifacts. Jobs are abstract closures scheduled by hosts under defined constraints, while many surrounding event-loop policies remain host-defined (ECMA International 2026). These statements directly motivate an abstraction layer rather than exposing goja stacks as if they were the specification.

27.2 Core calculi and operational models

Research semantics often desugar JavaScript into a smaller core calculus or give a direct operational semantics. The Essence of JavaScript develops a core calculus and desugaring approach; Maffeis, Mitchell, and Taly present an operational semantics intended to cover substantial language behavior (Maffeis, Mitchell, and Taly 2008; Guha, Saftoiu, and Krishnamurthi 2010). These models help define source-level event categories and reveal where surface constructs elaborate into common semantic operations.

27.3 Mechanized and executable JavaScript semantics

JSCert combines a mechanized specification with an extracted interpreter and validates it against test suites. KJS defines an executable semantics in K and uses symbolic execution and semantic coverage. JavaScript program logics reason about heaps, prototypes, property access, and dynamic features. These projects demonstrate that executable or mechanized JavaScript semantics can support both formal results and practical defect discovery (Bodin et al. 2014; Park, Stefanescu, and Rosu 2015; Gardner, Maffeis, and Smith 2012).

27.4 Double debugging and semantic coverage

JSExplain exposes the state of a specification-oriented interpreter alongside the interpreted program, enabling users to step through both levels (Chargueraud, Schmitt, and Wood 2018). A semantic probe workbench can generalize this idea by aligning three views: JavaScript source, formal rule or abstract operation, and goja VM event. KJS demonstrates the complementary value of semantic-rule coverage: tests can cover source lines while missing semantic cases. Event and rule IDs let coverage be reported over the semantics itself.

27.5 Specification evolution

The ECMAScript specification evolves yearly, and goja implements a particular feature set. Every semantic adapter should be versioned by specification edition and formal-model commit. Events whose meaning depends on newer features should declare feature gates. The system should not imply that one event schema completely captures all future ECMAScript behavior; it should support extension with compatibility tests and explicit deprecation.

28. Mapping goja execution to a semantic event ABI

goja is suitable because the interpreter and object model are implemented in Go, but correct hooks must cover all invocation and unwind paths.

28.1 Relevant goja structure

In the inspected goja source snapshot, the VM maintains the active Program and program counter, value stack, stack and argument indexes, lexical stash, call stack, iterator and reference stacks, try stack, new-target and result values, interrupt state, async runner, and profiler tracker. Instructions implement an internal `exec(vm)` interface, and the run loop dispatches the active instruction. This structure gives direct access to semantic transition points but also means the instrumentation API must be designed within goja or a maintained fork (Dop251 2026).

The Program representation stores internal instructions, a function name, source file, and source mapping. Public APIs support parsing, compiling, and running Programs. These are useful foundations for stable metadata, but the existing public API does not expose the full VM state or semantic hooks required by the proposal.

28.2 Calls and entries

Ordinary bytecode calls dispatch through a callee object's `vmCall` method. Interpreted functions switch VM context and begin executing a new Program; native functions call Go code synchronously and update the caller stack; Go code can also invoke JavaScript functions through external call paths. A complete event layer must route or wrap each path through one invocation lifecycle. Patching only the ordinary call instruction would miss external and internal invocation paths.

`CallBefore` should be emitted while caller arguments and receiver are stable. `FunctionEnter` should be emitted only after the function prologue establishes parameter and local state. A correlation record belongs to the saved caller context so that the eventual return or unwind can produce `CallAfter` or `CallThrow` even though the interpreted call did not complete synchronously.

28.3 Returns and unwinding

The inspected VM implements normal return by moving the result into the caller-facing stack slot, restoring the previous context, and advancing the caller program counter. Exception handling restores stack, stash, private environment, and call-stack state while searching try frames. Semantic exit events must execute before relevant frame information is discarded, and exception exit events must be generated for every crossed frame. This is the highest-risk correctness area because one abrupt completion can remove multiple frames (Dop251 2026).

28.4 Constructors, async runners, and generators

Constructors, super calls, async function runners, and generators have separate paths. The patch plan should first create internal common helpers for semantic invocation and completion, then migrate paths incrementally while preserving tests. Async settlement should be modeled separately from synchronous Promise return. Generator creation should be separated from body entry and later resume events.

28.5 Disabled-path design

The fast path should be designed before the dispatcher. For call-level events, one predictable event-mask check at the semantic boundary is the target. Instruction-level tracing should select a separate run loop when enabled, similar in shape to goja's specialized profiling path, rather than placing an unconditional generic callback around every instruction. Attachment selectors should already be resolved to compact IDs or bitsets.

Schematic hook shape. Exact stack access must follow goja internals and event contracts.

```
// schematic internal fast path
if vm.probes.eventMask & EventCallBefore != 0 {
  vm.probes.fireCallBefore(vm, callee, argc, siteID)
}
callee.self.vmCall(vm, argc)

// interpreted calls complete later through return or unwind;
// native calls may complete synchronously.
```

29. Stable metadata and the semantic map

Metadata is the bridge among source code, compiler decisions, VM execution, formal semantics, and research artifacts.

29.1 Program and site identifiers

ProgramID should derive from normalized source content, compiler options, language mode, and relevant dependency metadata. Within a Program, deterministic numbering can assign FunctionID, CallSiteID, BindingID, BranchSiteID, EffectSiteID, and JobOriginID. IDs need not be globally meaningful without the ProgramID; together they provide stable keys for probes, traces, coverage, and proof obligations.

29.2 Binding locations and liveness

Reading lexical locals is more difficult than reading arguments. A compiler may place a binding in a stack slot, stash, closure environment, or optimized representation; it may not be live at every site. The compiler should emit a binding-location table indexed by semantic site or program-counter range. `read_local(BindingID)` is valid only when the table says the binding is live and the capability permits it. This prevents probes from depending on raw stack indexes.

29.3 Mapping to specification operations and formal rules

A proof adapter can associate sites or macro-events with ECMA-262 abstract-operation identifiers and with rule identifiers from a Redex, K, Rocq, or Lean model. The mapping may be one-to-many: one VM macro-step can implement several specification algorithms, and one formal rule can span several VM sites. The metadata should allow ranges and hierarchical labels rather than force a false one-to-one relation.

29.4 Source maps and generated code

Goja supports source maps. The semantic map should preserve both generated and original spans, because performance and VM debugging may need generated positions while academic claims refer to source constructs. Dynamic code receives a synthetic origin chain: the event and source site that invoked eval, Function, or a module loader, plus a content hash of the generated source.

29.5 Metadata validation

Metadata is generated by a complex compiler and can be wrong. The toolchain should validate monotonic source ranges, site uniqueness, binding liveness consistency, instruction-range bounds, and hashes. Translation-validation checks can also replay simple stack-effect facts or compare event-site annotations against compiled control flow. Traces must include the metadata hash so that an event cannot be interpreted against a different build.

30. Daily academic tools and research workflows

The architecture succeeds only if it lowers the cost of routine semantics work: exploring a rule, finding a counterexample, aligning models, and packaging evidence.

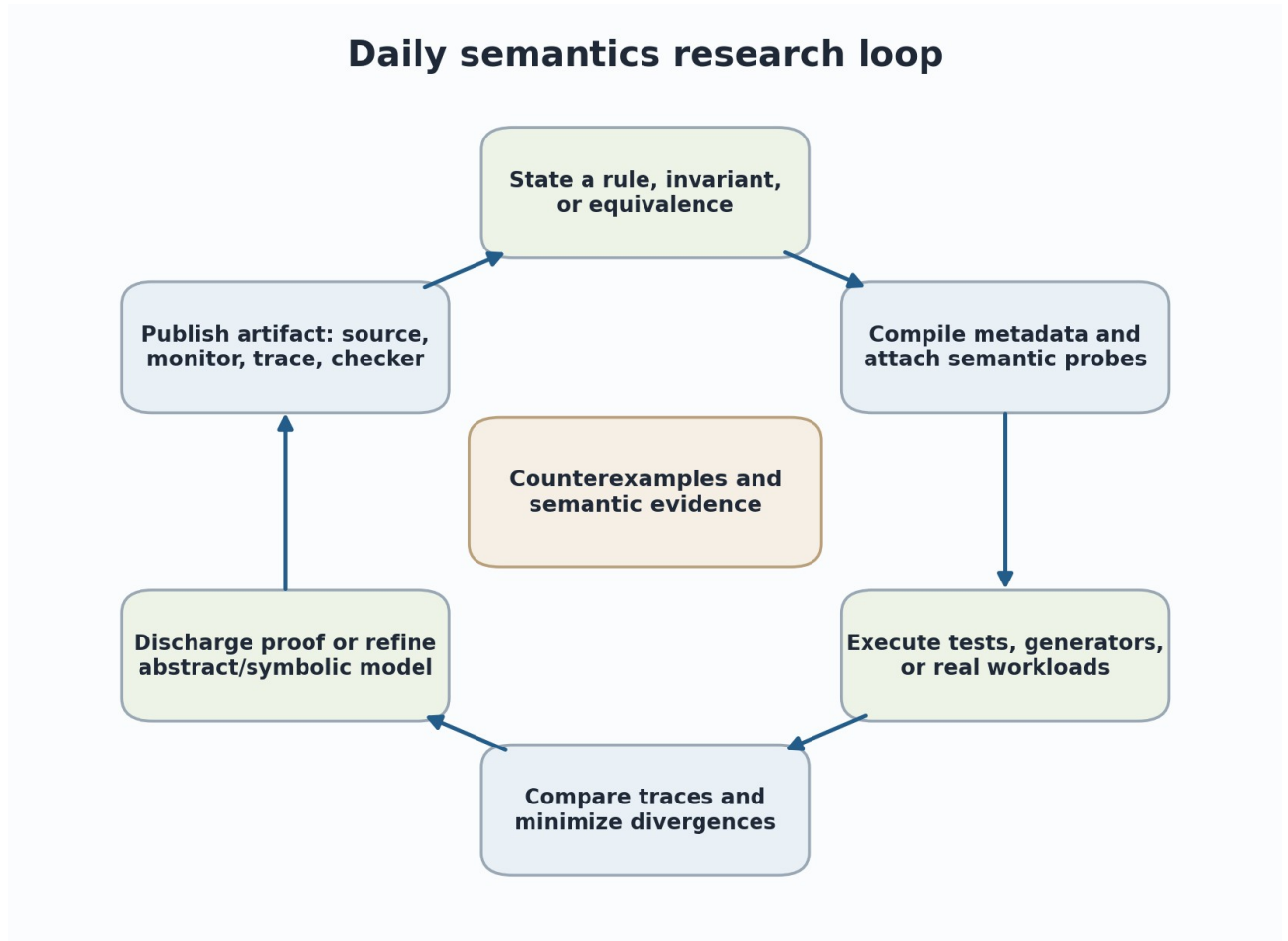


Figure 8. A daily loop from semantic hypothesis through execution, comparison, refinement, and artifact publication.

30.1 Semantic double debugger

The debugger should present synchronized panes for source code, formal rule or abstract operation, semantic event, monitor state, and selected VM projection. Stepping can be by source statement, semantic macro-event, formal rule, job boundary, or call frame. A divergence mode runs a reference semantics and goja together and stops at the first normalized event disagreement. This extends the double-debugger methodology of JSExplain to an implementation/reference comparison (Chargueraud, Schmitt, and Wood 2018).

30.2 Semantic coverage explorer

Coverage should include event cases, formal rules, specification operations, completion kinds, exceptional edges, proxy paths, Promise transitions, and native summary edges. A test suite can then answer: which return-through-finally cases were exercised; which constructor branches remain uncovered; whether every monitor transition has a witness; and whether a paper's claimed language subset has executable tests for each rule. Redex and KJS provide strong precedents for semantics-driven testing and coverage (Felleisen, Findler, and Flatt 2009; Park, Stefanescu, and Rosu 2015).

30.3 Invariant miner and candidate-lemma generator

Researchers can select semantic sites and value projections, run generated or corpus workloads, and infer likely invariants. Candidates appear with support counts, counterexamples, unknown-value rates, and site provenance. One click exports a candidate to the contract language, SMT-LIB, or a proof-assistant lemma template. The interface must label candidates as hypotheses until statically proved or otherwise justified (Ernst et al. 2001, 2007).

30.4 Proof debugger

A proof debugger attaches executable forms of induction hypotheses, simulation invariants, or representation relations. On violation it displays the smallest event prefix and projected configuration that falsifies the candidate. The user can

request test minimization, compare the formal and concrete states, and export the case. This is particularly useful before investing in long interactive proofs.

30.5 Abstract-analysis calibrator

The calibrator aligns concrete event observations with abstract states. It shows where the abstract state is precise, where it contains impossible values, and which join or summary introduced the imprecision. Concrete traces cannot justify removing abstract possibilities, but they guide domain design and regression tests. CEGAR can use solver-checked feasibility to turn this guidance into sound refinement.

30.6 Temporal protocol laboratory

A graphical or textual automaton editor defines parameterized protocols over semantic events. The tool generates the probe IR, an offline replay monitor, a property-based test generator, and theorem-prover definitions. Live views show monitor instances keyed by object, transaction, Promise, or job identity. Capacity and eviction status are visible so that monitoring gaps cannot be mistaken for success.

30.7 Artifact packager

The packager creates a directory or archive containing pinned sources, semantic metadata, monitor sources and IR, native summaries, input corpus or generator seeds, traces or aggregate certificates, benchmark configuration, and an independent checker. A paper can cite a query that recomputes a table or reproduces a divergence. This makes semantic claims inspectable without requiring the reader to trust screenshots or manually reconstructed logs.

Table 12. Proposed daily research tools.

Tool	Primary input	Primary output
Semantic double debugger	Program, model adapter, live or replay trace	Aligned source/rule/VM state and first divergence.
Coverage explorer	Metadata, traces, rule inventory	Uncovered semantic cases and exercised obligations.
Invariant miner	Selected sites and value projections	Candidate contracts with provenance and counterexamples.
Proof debugger	Candidate invariant or simulation relation	Minimal concrete violation and exported proof case.
Analysis calibrator	Abstract states plus concrete traces	Precision diagnostics and refinement candidates.
Protocol laboratory	Temporal or automaton property	Verified runtime monitor, replay monitor, test generator.
Artifact packager	Manifest, sources, probes, traces, queries	Reproducible research bundle.

31. Worked example: a ledger contract and protocol monitor

A single example illustrates operational events, frame-local state, static residualization, temporal monitoring, symbolic counterexamples, and proof artifacts.

31.1 JavaScript program

Running JavaScript example.

```
class Ledger {
  constructor(balance) {
    this.balance = balance;
    this.open = true;
  }

  debit(amount) {
    if (!this.open) throw new Error("closed");
    if (amount < 0) throw new Error("negative amount");
  }
}
```



```

    this.balance -= amount;
    return this.balance;
}

close() {
    this.open = false;
}
}

```

The intended properties are: initial balance is nonnegative; debit receives a nonnegative amount; a successful debit preserves nonnegative balance and decreases it exactly by amount; debit on a closed ledger throws without changing balance; close is irreversible; and no debit call may occur after a successful close.

31.2 Contract probe

Illustrative source language for an entry/return/throw contract.

```

probe LedgerDebit {
    attach function("ledger.js", "Ledger.debit");

    frame_local<i64> entry_balance;
    frame_local<bool> entry_open;

    on enter(amount) {
        assert as_i64(amount) >= 0;
        entry_balance = own_i64(this, "balance");
        entry_open = own_bool(this, "open");
        assert entry_open;
    }

    on return(amount, result) {
        let current = own_i64(this, "balance");
        assert current >= 0;
        assert current == entry_balance - as_i64(amount);
        assert as_i64(result) == current;
    }

    on throw(error) {
        assert own_i64(this, "balance")? == entry_balance;
    }
}

```

The compiler resolves the function selector to FunctionID, checks that requested property helpers are safe own-data-property reads, allocates fixed frame-local slots, and emits event-specific IR. Enter records the pre-state; Return relates it to the post-state; Throw checks exceptional frame conditions. The monitor receives no arbitrary object traversal or JavaScript callbacks.

31.3 Protocol monitor

Parametric typestate monitor keyed by stable ledger identity.

```

state map<ObjectID, State> ledger_state capacity 10000 fail_closed;

enum State { OPEN, CLOSED }

on ConstructAfter(instance) where function == Ledger.constructor {
    state[object_id(instance)] = OPEN;
}

on FunctionReturn(this) where function == Ledger.close {
    require state[object_id(this)] == OPEN;
    state[object_id(this)] = CLOSED;
}

on CallBefore(this) where callee == Ledger.debit {
    require state[object_id(this)] == OPEN;
}

```

This property is a safety protocol. The first debit after close is a finite bad prefix. The state map must be non-evicting or fail closed; otherwise an evicted CLOSED state could allow a later debit. Object identity lifecycle needs an explicit policy. If ledger objects can be collected, reclamation may delete terminal CLOSED instances, but the monitor should record that reclamation event for replay.

31.4 Static and symbolic interpretations

An interval or refinement analysis may prove $\text{amount} \geq 0$ at selected call sites, allowing the corresponding entry assertion to be erased there. It may prove that close is called at most once under a closed-world module analysis. Unresolved dynamic call sites retain the monitor. A symbolic executor can negate current ≥ 0 and search for an input balance and amount that cause a violation. If constructor balance is unconstrained, the solver immediately finds a negative initial balance, revealing a missing constructor contract.

31.5 Logical formulation

```
entry_open and amount >= 0 and balance = b and debit returns r => r = b - amount and
r >= 0
```

One functional postcondition; a complete model must also account for the branch that throws.

In a proof assistant, LedgerDebit becomes a relation between entry and exit states at events with the selected FunctionID. The monitor-compiler theorem states that the IR reaches Violation exactly when the trace prefix violates the executable contract semantics, subject to helper correctness. A separate program theorem proves that all modeled executions satisfy the contract. Runtime traces provide regression cases and evidence for actual executions but are not substituted for the program theorem.

31.6 Counterexample trace

Table 13. Example bad prefix for a ledger invariant.

Seq	Event	Key observation	Monitor transition
41	ConstructAfter	ledger O17, balance=5, open=true	state[O17] := OPEN
58	FunctionEnter debit	amount=8, balance=5	entry_balance := 5; precondition holds
63	PropertyWrite balance	new value=-3	optional effect trace
66	FunctionReturn debit	result=-3, balance=-3	postcondition current>=0 fails
67	Violation	LedgerDebit/postcondition-1	execution stopped; trace slice sealed

The minimizer preserves the semantic path and reduces constructor balance and debit amount while retaining the violation. The artifact includes the source, metadata, probe IR, five-event trace slice, helper projections, and checker output. The same case becomes a unit test for the runtime event mapping, a solver regression, and a proof-assistant counterexample to an overly strong theorem that omitted the balance precondition.

32. Evaluation methodology

Evaluation must treat semantic fidelity, safety, performance, research utility, and reproducibility as separate axes.

32.1 Semantic conformance suite

Build a focused corpus for each event family. Calls cover ordinary, arrow, bound, proxy, native, Go-to-JavaScript, recursive, and constructor calls. Completion covers implicit return, explicit return, return through finally, caught and uncaught throws, and multi-frame unwinds. Async covers await fulfillment, rejection, nested awaits, handler absence, job ordering, cancellation, and host callbacks. Each test declares the expected event partial order and observation fields.

Generated testing should operate over the formal subset and compare goja with a Redex, K, or extracted model. Semantic-rule coverage identifies untested rules. Mutation testing can deliberately move or omit hooks to measure whether the suite detects mapping defects. This is stronger than merely checking that probes fire.

32.2 Verifier evaluation

The verifier test suite should include accepted programs, type confusion attempts, invalid event-phase access, lifetime escape, excessive emission, map-schema mismatch, branch-join errors, unsupported loops, helper capability violations, and malformed binaries. Fuzz the decoder and verifier with structure-aware generators. Compare verifier decisions with a small reference semantics of the IR. Measure verification latency and worst-case path growth.

32.3 Performance experiments

Benchmark micro-level events and application workloads. Report nanoseconds or relative overhead per call, return, throw, property effect, job event, and instruction event. Separate interpreted and native calls. Include recursive depth, exception-heavy code, Promise workloads, host-blocking calls, and module-heavy applications. Measure allocation, garbage collection, ring-buffer loss, and map memory in addition to elapsed time.

Table 14. Required performance configurations.

Configuration	Purpose
Baseline	Unmodified or instrumentation-compiled-but-disabled interpreter.
Event mask disabled	Cost of retaining probe fields and branch structure.
Family enabled, no attachment	Dispatcher/site-filter cost.
No-op verified probe	Invocation and verifier-runtime framework cost.
Counter/histogram	Representative metrics aggregation.
Frame-local contract	Representative invariant monitor.
Trace emission	Serialization/ring-buffer and offline drain cost.
Instruction sampling	High-frequency optional diagnostic tier.

32.4 Research-utility study

Select published or realistically sized semantic models and seed known defects: wrong exception boundary, missing constructor case, incorrect job order, unsound property helper, or incomplete native summary. Ask participants to locate the defect using ordinary logs versus the semantic workbench. Measure time to first relevant divergence, counterexample size, number of irrelevant steps inspected, and correctness of diagnosis. A smaller within-subject study can provide initial evidence; a larger study is required for broad productivity claims.

32.5 Reproducibility and artifact evaluation

Every reported table should be derivable from a pinned manifest and query. Traces should either be included or regenerable from seeds and recorded host oracles. The checker should validate schema, hashes, event loss, and monitor transitions before computing results. Performance artifacts should record machine, Go version, operating system, CPU governor, load, and warm-up policy. The paper should distinguish deterministic semantic evidence from noisy timing data.

33. Implementation roadmap and open research questions

The roadmap should establish event correctness before investing heavily in a rich probe language or broad event coverage.

33.1 Phase 1: semantic lifecycle events

1. Fork or extend goja with a private event manager and fast event masks.
2. Implement CallBefore, FunctionEnter, FunctionReturn, FunctionThrow, CallAfter, CallThrow, ConstructBefore, ConstructAfter, ConstructThrow, HostEnter, HostReturn, and HostThrow.
3. Add stable ProgramID, FunctionID, CallSiteID, FrameID, and source spans.

4. Use a trusted Go sink first; do not build the probe bytecode until event order is tested.
5. Create exhaustive tests for normal, native, external, proxy, recursive, constructor, finally, and unwind paths.

33.2 Phase 2: verified observational probes

6. Implement acyclic typed IR, register verifier, primitive projections, safe own-data-property helpers, counters, hashes, frame-local state, and bounded ring-buffer emission.
7. Integrate a manager through go-go-goja RuntimeInitializer and runtime-owner scheduling.
8. Add attach/detach control-plane operations, manifests, and strict versus best-effort modes.
9. Benchmark disabled and common enabled paths before adding higher-frequency events.

33.3 Phase 3: semantics adapters and offline workbench

10. Define normalized event formats for a small Redex, K, or proof-assistant model.
11. Build differential trace comparison, semantic coverage, minimization, and replay.
12. Add property-language compilation to runtime IR and offline monitor.
13. Export SMT-LIB and theorem-prover definitions with a manifest of assumptions.

33.4 Phase 4: async, generators, properties, and residual verification

14. Add job, Promise, await, generator, and causal events with continuation-aware storage.
15. Add carefully stratified property-effect events and safe object-shape observations.
16. Implement one abstract domain and one symbolic execution adapter for the shared IR.
17. Add static discharge and residual-monitor manifests.
18. Evaluate hyperproperty and multi-run trace alignment separately from single-run monitoring.

33.5 Open theoretical questions

- What is the smallest event basis that is adequate for a useful class of JavaScript analyses?
- Can read-only probes be proved transparent in a host model that includes observable time, memory pressure, interrupts, and Promise scheduling?
- Which event capabilities admit a full-abstraction result, and which necessarily expose implementation representation?
- How should safe observation of proxies, accessors, weak references, and host objects be modeled?
- Can monitor fusion be proved correct while preserving independent budgets and verdict ordering?
- How can dynamic code and native modules be enrolled without making the trusted base impractically large?
- Can proof-relevant traces remain compact enough for workload-scale use while supporting independent checking?
- What static domains best complement runtime probes for JavaScript contracts and async protocols?
- How should event schemas evolve across ECMAScript editions and interpreter refactorings?
- Can semantic probes support translation validation of interpreter optimizations or alternative bytecode compilers?

33.6 Final assessment

The project is technically plausible and theoretically rich because it creates an explicit seam between concrete interpreter execution and formal interpretation. Its novelty does not lie in inventing runtime monitoring, executable semantics, or safe extension in isolation. It lies in combining them around a versioned semantic event ABI inside a practical interpreter, with a single verified property substrate and research-facing metadata. The decisive risk is event semantics: if events are incomplete, mistimed, or ambiguously projected, the rest of the system can produce precise answers to the wrong questions.

The recommended strategy is therefore conservative. Begin with a small event subset, state its semantics, test every control path, prove erasure in a model, and make traces independently checkable. Once that foundation is reliable, metrics, contracts, abstract interpretation, symbolic execution, temporal monitoring, and proof tools can share it. The result can become both an instrumentation facility and a semantics laboratory used during ordinary academic work rather than only in final artifact construction.

Appendices

Formal sketches, schemas, implementation interfaces, and bibliography.

Appendix A. Formal core for a first mechanization

A.1 Core configurations

A first mechanization should use a deliberately small language with values, variables, mutable objects, function calls, constructors, return, throw, try/catch/finally, and native effects represented as abstract events. The purpose is to prove the instrumentation architecture, not to formalize all of ECMAScript immediately. A CESK-like machine makes the environment, store, and continuation explicit.

Schematic core language and machine components.

```
Value v    ::= n | b | string | undefined | null | location | closure
Expr e     ::= v | x | e(e*) | new e(e*) | e.p | e.p := e
           | return e | throw e | try e catch(x) e finally e

Env rho : Variable -> Address
Store sigma : Address -> Storable
Kont kappa ::= halt | callK(...) | returnK(...) | throwK(...) | finallyK(...)
Config C ::= <e, rho, sigma, kappa> | <completion, sigma, kappa>
```

A.2 Event-labelled rules

Rules should distinguish call-site and callee lifecycle events. The following are schemas rather than finished inference rules. Observations are projections and therefore do not contain the full configuration.

Illustrative event-labelled transition schemas.

```
CALL-BEFORE
  evaluate callee, receiver, and arguments
  -----
  <callsite, rho, sigma, kappa>
    -[CallBefore(site, callee, receiver, args)]->
  <callee-body, rho_f, sigma, callK(frame, kappa)>

FUNCTION-RETURN
  final completion is Normal(v)
  -----
  <Normal(v), sigma, callK(frame, kappa)>
    -[FunctionReturn(frame, v)]->
  <caller-result(v), sigma, kappa>

FUNCTION-THROW
  abrupt completion crosses frame
  -----
  <Throw(x), sigma, callK(frame, kappa)>
    -[FunctionThrow(frame, x)]->
  <Throw(x), sigma, kappa>
```

A.3 Instrumented product

Core monitor-product semantics.

```
Program transition:
  C -[e/o]-> C'

Monitor transition:
  delta(m, e, o) = (m', action)

Product transition:
  (C, m) ==> (C', m')          when action = Continue
  (C, m) ==> Violation(id, t)  when action = Violate(id)
```

```
Silent transition:
  C -[tau]-> C'
  -----
  (C, m) ==> (C', m)
```

A.4 Candidate theorems

- Probe progress: a verified probe invocation either completes within its budget or produces a defined budget outcome; it does not become stuck.
- Probe preservation: executing a verified instruction preserves the verifier's register, stack, handle-lifetime, and map-schema invariants.
- Observational erasure: for observe-mode probes, erasing monitor state and tau steps yields an execution related to the uninstrumented execution.
- Event soundness: every non-tau event in the instrumented machine corresponds to the event-labelled source rule of the same kind.
- Event completeness for the core: every selected source rule produces exactly the required event when the family is enabled.
- Monitor compiler correctness: compiling a core contract or automaton produces an IR monitor with the same bad prefixes.
- Trace checker soundness: acceptance by the checker implies that local event and monitor-transition constraints hold for the supplied trace.

Appendix B. Event schema sketch

B.1 Common envelope

Common trace envelope. Runtime probe contexts should use event-specific typed records.

```
EventEnvelope {
  schema_version : u32
  run_id         : RunID
  sequence       : u64
  event_id       : EventID
  kind           : EventKind

  program_id     : ProgramID?
  function_id    : FunctionID?
  site_id        : SiteID?
  frame_id       : FrameID?
  continuation_id : ContinuationID?
  job_id         : JobID?

  source         : SourceSpan?
  formal_rule    : RuleID?
  spec_operation : OperationID?
  cause          : EventID?

  payload        : EventPayload
}
```

B.2 Event payload examples

Table 15. Event payload contracts for an initial ABI.

Event	Required payload	Semantic phase
CallBefore	callee, receiver, argument view, call kind, call-site ID	After operand evaluation; before invocation semantics.
FunctionEnter	function, frame, this availability, parameters, new target	After prologue/parameter initialization.

Event	Required payload	Semantic phase
FunctionReturn	function, frame, final normal result	Immediately before crossing callee boundary.
FunctionThrow	function, frame, final exception value/category	When unwinding crosses the frame.
CallAfter	call site, correlated frame/call ID, caller-visible result	After invocation completes normally.
ConstructAfter	constructor, new target, final instance	After constructor return rules choose the instance.
AwaitSuspend	frame, continuation, awaited Promise/value, reaction/job origin	After continuation capture and reaction setup.
AwaitResume	continuation, job, fulfilled/rejected input	At resumption before body continues.
PropertyReadSafe	base ID, key ID/hash, data value or Unknown reason	Pure observation helper; not ordinary JavaScript Get.
HostCall	module/function summary ID, bounded arguments, effect token	At declared host boundary.

B.3 Loss and sampling records

Every stream should carry explicit loss metadata. EvidenceLost includes the event family, first missing sequence if known, count, reason, and whether the run remains valid for each attached analysis. SamplingConfig identifies the selection process and seed. Strict monitors reject loss; statistical consumers may continue with declared estimators.

Appendix C. Probe IR and verifier sketch

C.1 Instruction classes

Table 16. Proposed IR instruction classes.

Class	Examples	Notes
Pure arithmetic	const, add, subtract, compare, boolean operations	Fixed width with defined overflow policy.
Control flow	conditional branch, bounded switch, return	Acyclic in version 1.
Context access	event_kind, site_id, arg_count, arg_handle, result_handle	Availability depends on event type and phase.
Value projection	typeof, as_i64, bool_copy, string_hash, object_id	No JavaScript coercion unless capability explicitly allows it.
Safe structure	own_data_property, descriptor_kind, shape_id	Returns Unknown for proxies/accessors/exotics.
Maps	lookup, insert, delete, counter_add, histogram_observe	Statically declared schema and capacity.
Frame state	frame_load, frame_store	Automatically scoped to semantic frame/continuation.
Output	emit fixed record, report, violation	Bounded record size; intervention excluded from observe mode.

C.2 Abstract verifier state

Schematic abstract state propagated by the verifier.

```

VerifierState {
  pc           : InstructionIndex
  registers    : Vector<RegAbstractType>
}

```

```

stack_height      : Interval
handle_lifetimes  : Map<Register, Lifetime>
budget            : CostInterval
emitted_bytes     : Interval
map_effects       : Set<MapCapability>
mode              : Observe | Enforce | Intervene
}

```

join(s1, s2) must preserve initialization and compatible types;
otherwise the program is rejected.

C.3 Helper contract example

Example helper contract suitable for verifier and formal modeling.

```

helper own_i64_property(
  obj : ValueHandle<EventLifetime>,
  property : PropertyID
) -> Result<i64, ObservationError>

```

Preconditions:

- current event grants ObjectReadSafe
- obj belongs to current runtime and event lifetime
- property is pre-resolved or bounded

Effects:

- no JavaScript calls, coercions, getters, proxies, or host calls
- no persistent references to obj
- bounded constant or declared shape-dependent cost

Result:

- Ok(n) only for an own data property containing an exact supported integer
- Err(Absent | Accessor | Proxy | Exotic | TypeMismatch | Budget) otherwise

Appendix D. go-go-goja integration and command sketches

D.1 Runtime initialization

Illustrative integration through the existing runtime-initializer boundary.

```

factory, err := engine.NewRuntimeFactoryBuilder().
  WithRuntimeInitializers(
    probe.Initializer(probe.Config{
      MaxProbeSteps: 4096,
      MaxMapBytes:   16 << 20,
      RingBytes:     4 << 20,
      AllowEnforce:  true,
      AllowMutation: false,
    })),
  ).
  Build()

rt, err := factory.NewRuntime(
  engine.WithStartupContext(ctx),
  engine.WithLifetimeContext(ctx),
)
defer rt.Close(ctx)

```

The initializer creates a per-runtime manager, attaches it to the underlying goja runtime, stores the manager in runtime-scoped values, registers shutdown behavior, and optionally installs a trusted host control module. Attach and detach operations execute through the runtime owner so that they respect goja's single-owner execution rule (Go-Go-Golems 2026; Dop251 2026).

D.2 Command-line workflow

Illustrative academic workbench commands.

```
# Verify and inspect a probe
xgoja-sem probe verify contracts.sprobe
xgoja-sem probe disassemble contracts.sprobe

# Execute with exact proof-mode tracing
xgoja-sem trace --schema ecma-2026 --probe contracts.sprobe --strict-evidence --output
run.semtrace program.js

# Compare with a reference semantics
xgoja-sem diff --left goja:run.semtrace --right k:model.semtrace --profile language-semantic

# Explore and export
xgoja-sem coverage --kind rule run.semtrace
xgoja-sem monitor --property promise-lifecycle.rv run.semtrace
xgoja-sem export --format rocq --slice violation:17 run.semtrace
```

D.3 Repository layout

One possible package layout.

```
goja fork or instrumentation branch/
probe_api.go
probe_events.go
probe_context.go
probe_metadata.go
probe_maps.go
probe_verifier.go
probe_ir.go

go-go-goja/
pkg/semanticprobe/
  initializer.go
  manager.go
  manifest.go
  exporter.go
  replay.go
cmd/xgoja-sem/
  trace.go
  diff.go
  coverage.go
  monitor.go
  export.go
```

Appendix E. Research-to-design crosswalk

Table 17. Literature crosswalk from theory to implementation decision.

Research area	Key references	Direct design consequence
Structural operational semantics	Plotkin 2004; Winskel 1993	Define event-labelled configurations and macro-transition rules.
Bisimulation and equivalence	Hennessey and Milner 1985; Sangiorgi 2011; Pitts 1997	State transparency as a weak or contextual equivalence relative to event capabilities.
Denotational effects	Moggi 1991; Plotkin and Power 2003; Plotkin and Pretnar 2009	Treat events as an effect signature and analyses as handlers.
Interaction trees	Xia et al. 2020	Represent recursive interactive behavior with executable event trees and weak bisimulation.
Runtime verification	Leucker and Schallhart 2009; Bauer et al. 2010/2011	Use explicit finite-prefix verdicts and automata/temporal monitor semantics.

Research area	Key references	Direct design consequence
Parametric monitoring	Rosu and Chen 2012; Havelund et al. 2018	Typed identity maps and trace slicing for per-object or per-request properties.
Contracts and hybrid checking	Findler and Felleisen 2002; Flanagan 2006; Nguyen et al. 2014	Compile pre/postconditions; statically discharge obligations and retain residual probes.
Abstract interpretation	Cousot and Cousot 1977; Cousot 2021; Van Horn and Might 2010	Implement verifier and static interpretation as sound abstract semantics.
Symbolic execution and CEGAR	King 1976; Clarke et al. 2000	Generate counterexamples and refine abstractions at stable semantic sites.
Hyperproperties	Clarkson and Schneider 2010; Finkbeiner et al. 2017	Add multi-run trace alignment for relational security and equivalence.
Safe extensibility	Wahbe et al. 1993; Nacula 1997; Morrisett et al. 1999	Restricted IR, capability helpers, proof/checker separation, and bounded authority.
BPF / DTrace	McCanne and Jacobson 1993; Cantrill et al. 2004; Linux docs 2026	Stable providers, verified programs, maps, aggregations, and fast disabled paths.
Executable semantics	Felleisen et al. 2009; Bodin et al. 2014; Park et al. 2015	Differential semantic traces, rule coverage, and model-driven testing.
Invariant mining	Ernst et al. 2001/2007; Nimmer and Ernst 2001	Generate candidate contracts with provenance, then statically verify or monitor them.
Verified compilation	Leroy 2009; Pnueli et al. 1998	Use refinement targets and per-program translation validation for semantic metadata.

References

URLs and DOI links identify editions used for this synthesis. Repository references are pinned to inspected source snapshots. The bibliography emphasizes primary papers, monographs, official specifications, and official kernel documentation.

- Alpern, B., and Schneider, F. B. (1985). Defining liveness. *Information Processing Letters*, 21(4), 181-185. [https://doi.org/10.1016/0020-0190\(85\)90056-0](https://doi.org/10.1016/0020-0190(85)90056-0)
- Bartocci, E., and Falcone, Y. (Eds.). (2018). *Lectures on Runtime Verification*. Springer. <https://doi.org/10.1007/978-3-319-75632-5>
- Bauer, A., Leucker, M., and Schallhart, C. (2010). Comparing LTL semantics for runtime verification. *Journal of Logic and Computation*, 20(3), 651-674. <https://doi.org/10.1093/logcom/exn075>
- Bauer, A., Leucker, M., and Schallhart, C. (2011). Runtime verification for LTL and TLTL. *ACM Transactions on Software Engineering and Methodology*, 20(4), Article 14. <https://doi.org/10.1145/2000799.2000800>
- Bodin, M., Chargueraud, A., Filaretti, D., Gardner, P., Maffei, S., Naudziuniene, D., Schmitt, A., and Smith, G. (2014). A trusted mechanised JavaScript specification. *Proceedings of POPL 2014*. <https://doi.org/10.1145/2535838.2535876>
- Cantrill, B. M., Shapiro, M. W., and Leventhal, A. H. (2004). Dynamic instrumentation of production systems. *USENIX Annual Technical Conference*. <https://www.usenix.org/legacy/publications/library/proceedings/usenix04/tech/general/cantrill.html>
- Chargueraud, A., Schmitt, A., and Wood, T. (2018). JSExplain: a double debugger for JavaScript. *Companion Proceedings of The Web Conference 2018*. <https://doi.org/10.1145/3184558.3185969>
- Chen, F., and Rosu, G. (2003). Towards monitoring-oriented programming: a paradigm combining specification and implementation. *Electronic Notes in Theoretical Computer Science*, 89(2), 108-127. [https://doi.org/10.1016/S1571-0661\(04\)81045-4](https://doi.org/10.1016/S1571-0661(04)81045-4)
- Chen, F., and Rosu, G. (2007). MOP: an efficient and generic runtime verification framework. *Proceedings of OOPSLA 2007*. <https://doi.org/10.1145/1297027.1297069>
- Claessen, K., and Hughes, J. (2000). QuickCheck: a lightweight tool for random testing of Haskell programs. *Proceedings of ICFP 2000*. <https://doi.org/10.1145/351240.351266>
- Clarke, E. M., Grumberg, O., Jha, S., Lu, Y., and Veith, H. (2000). Counterexample-guided abstraction refinement. *Proceedings of CAV 2000*. https://doi.org/10.1007/10722167_15
- Clarkson, M. R., and Schneider, F. B. (2010). Hyperproperties. *Journal of Computer Security*, 18(6), 1157-1210. <https://doi.org/10.3233/JCS-2009-0393>
- Clarkson, M. R., Finkbeiner, B., Koleini, M., Micinski, K. K., Rabe, M. N., and Sanchez, C. (2014). Temporal logics for hyperproperties. *Proceedings of POST 2014*. https://doi.org/10.1007/978-3-642-54792-8_15

- Cousot, P. (2021). Principles of Abstract Interpretation. MIT Press. <https://mitpress.mit.edu/9780262044905/principles-of-abstract-interpretation/>
- Cousot, P., and Cousot, R. (1977). Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. Proceedings of POPL 1977. <https://doi.org/10.1145/512950.512973>
- De Giacomo, G., and Vardi, M. Y. (2013). Linear temporal logic and linear dynamic logic on finite traces. Proceedings of IJCAI 2013. <https://www.ijcai.org/Proceedings/13/Papers/099.pdf>
- Dop251. (2026). goja: ECMAScript implementation in pure Go. Source snapshot 0fc1d42c1dc933899bb84267b06e3f64b6c899ef. <https://github.com/dop251/goja/tree/0fc1d42c1dc933899bb84267b06e3f64b6c899ef>
- ECMA International. (2026). ECMAScript 2026 Language Specification, ECMA-262. <https://tc39.es/ecma262/2026/>
- Ernst, M. D., Cockrell, J., Griswold, W. G., and Notkin, D. (2001). Dynamically discovering likely program invariants to support program evolution. IEEE Transactions on Software Engineering, 27(2), 99-123. <https://doi.org/10.1109/32.908957>
- Ernst, M. D., Perkins, J. H., Guo, P. J., McCamant, S., Pacheco, C., Tschantz, M. S., and Xiao, C. (2007). The Daikon system for dynamic detection of likely invariants. Science of Computer Programming, 69(1-3), 35-45. <https://doi.org/10.1016/j.scico.2007.01.015>
- Felleisen, M., Findler, R. B., and Flatt, M. (2009). Semantics Engineering with PLT Redex. MIT Press. <https://mitpress.mit.edu/9780262062756/semantics-engineering-with-plt-redex/>
- Findler, R. B., and Felleisen, M. (2002). Contracts for higher-order functions. Proceedings of ICFP 2002. <https://doi.org/10.1145/581478.581484>
- Finkbeiner, B., Hahn, C., Stenger, M., and Tentrup, L. (2017). Monitoring hyperproperties. Proceedings of Runtime Verification 2017. https://doi.org/10.1007/978-3-319-67531-2_12
- Flanagan, C. (2006). Hybrid type checking. Proceedings of POPL 2006. <https://doi.org/10.1145/1111037.1111059>
- Gardner, P., Maffei, S., and Smith, G. D. (2012). Towards a program logic for JavaScript. Proceedings of POPL 2012. <https://doi.org/10.1145/2103656.2103663>
- Go-Go-Golems. (2026). go-go-goja: native-module playground and runtime composition for goja. Source snapshot ae413f594d0ed4e36123ac925e195666e0f0e759. <https://github.com/go-go-golems/go-go-goja/tree/ae413f594d0ed4e36123ac925e195666e0f0e759>
- Greenberg, M., Pierce, B. C., and Weirich, S. (2010). Contracts made manifest. Proceedings of POPL 2010. <https://doi.org/10.1145/1707801.1706341>
- Guha, A., Saftoiu, C., and Krishnamurthi, S. (2010). The essence of JavaScript. Proceedings of ECOOP 2010. https://doi.org/10.1007/978-3-642-14107-2_7
- Gunter, C. A. (1992). Semantics of Programming Languages: Structures and Techniques. MIT Press. <https://mitpress.mit.edu/9780262570954/semantics-of-programming-languages/>
- Havelund, K., Reger, G., Thoma, D., and Zalinescu, E. (2018). Monitoring events that carry data. In Lectures on Runtime Verification. https://doi.org/10.1007/978-3-319-75632-5_3
- Hennessy, M., and Milner, R. (1985). Algebraic laws for nondeterminism and concurrency. Journal of the ACM, 32(1), 137-161. <https://doi.org/10.1145/2455.2460>
- Keil, M., and Thiemann, P. (2015). TreatJS: higher-order contracts for JavaScript. Proceedings of ECOOP 2015. <https://doi.org/10.4230/LIPIcs.ECOOP.2015.28>
- Kiczales, G., Lamping, J., Mendhekar, A., Maeda, C., Lopes, C. V., Loingtier, J.-M., and Irwin, J. (1997). Aspect-oriented programming. Proceedings of ECOOP 1997. <https://doi.org/10.1007/BFb0053381>
- King, J. C. (1976). Symbolic execution and program testing. Communications of the ACM, 19(7), 385-394. <https://doi.org/10.1145/360248.360252>
- Klein, C., Clements, J., Dimoulas, C., Eastlund, C., Felleisen, M., Flatt, M., McCarthy, J. A., Rafkind, J., Tobin-Hochstadt, S., and Findler, R. B. (2012). Run your research: on the effectiveness of lightweight mechanization. Proceedings of POPL 2012. <https://doi.org/10.1145/2103656.2103691>
- Lamport, L. (1978). Time, clocks, and the ordering of events in a distributed system. Communications of the ACM, 21(7), 558-565. <https://doi.org/10.1145/359545.359563>
- Leroy, X. (2009). Formal verification of a realistic compiler. Communications of the ACM, 52(7), 107-115. <https://doi.org/10.1145/1538788.1538814>
- Leucker, M., and Schallhart, C. (2009). A brief account of runtime verification. Journal of Logic and Algebraic Programming, 78(5), 293-303. <https://doi.org/10.1016/j.jlap.2008.08.004>
- Ligatti, J., Bauer, L., and Walker, D. (2005). Edit automata: enforcement mechanisms for run-time security policies. International Journal of Information Security, 4(1-2), 2-16. <https://doi.org/10.1007/s10207-004-0046-8>
- Linux Kernel Documentation. (2026). eBPF verifier, maps, and program safety documentation. <https://docs.kernel.org/bpf/>
- Maffei, S., Mitchell, J. C., and Taly, A. (2008). An operational semantics for JavaScript. Proceedings of APLAS 2008. https://doi.org/10.1007/978-3-540-89330-1_22

- McCanne, S., and Jacobson, V. (1993). The BSD packet filter: a new architecture for user-level packet capture. USENIX Winter Conference. <https://www.tcpdump.org/papers/bpf-usenix93.pdf>
- Milner, R. (1977). Fully abstract models of typed lambda-calculi. *Theoretical Computer Science*, 4(1), 1-22. [https://doi.org/10.1016/0304-3975\(77\)90053-6](https://doi.org/10.1016/0304-3975(77)90053-6)
- Moggi, E. (1991). Notions of computation and monads. *Information and Computation*, 93(1), 55-92. [https://doi.org/10.1016/0890-5401\(91\)90052-4](https://doi.org/10.1016/0890-5401(91)90052-4)
- Morrisett, G., Walker, D., Crary, K., and Glew, N. (1999). From System F to typed assembly language. *ACM Transactions on Programming Languages and Systems*, 21(3), 527-568. <https://doi.org/10.1145/319301.319345>
- Mousavi, M. R., Reniers, M. A., and Groote, J. F. (2007). SOS formats and meta-theory: 20 years after. *Theoretical Computer Science*, 373(3), 238-272. <https://doi.org/10.1016/j.tcs.2006.12.019>
- Necula, G. C. (1997). Proof-carrying code. *Proceedings of POPL 1997*. <https://doi.org/10.1145/263699.263712>
- Nguyen, P. C., Tobin-Hochstadt, S., and Van Horn, D. (2014). Soft contract verification. *Proceedings of ICFP 2014*. <https://doi.org/10.1145/2628136.2628156>
- Nielson, F., Nielson, H. R., and Hankin, C. (1999). *Principles of Program Analysis*. Springer. <https://doi.org/10.1007/978-3-662-03811-6>
- Nielson, H. R., and Nielson, F. (2007). *Semantics with Applications: An Appetizer*. Springer. <https://doi.org/10.1007/978-1-84628-692-6>
- Nimmer, J. W., and Ernst, M. D. (2001). Static verification of dynamically detected program invariants. *Electronic Notes in Theoretical Computer Science*, 55(2), 255-276. [https://doi.org/10.1016/S1571-0661\(04\)00256-7](https://doi.org/10.1016/S1571-0661(04)00256-7)
- Park, D., Stefanescu, A., and Rosu, G. (2015). KJS: a complete formal semantics of JavaScript. *Proceedings of PLDI 2015*. <https://doi.org/10.1145/2737924.2737991>
- Pierce, B. C. (2002). *Types and Programming Languages*. MIT Press. <https://mitpress.mit.edu/9780262162098/types-and-programming-languages/>
- Pitts, A. M. (1997). Operationally-based theories of program equivalence. In *Semantics and Logics of Computation*. <https://doi.org/10.1017/CBO9780511526619.007>
- Plotkin, G. D. (1977). LCF considered as a programming language. *Theoretical Computer Science*, 5(3), 223-255. [https://doi.org/10.1016/0304-3975\(77\)90044-5](https://doi.org/10.1016/0304-3975(77)90044-5)
- Plotkin, G. D. (2004). A structural approach to operational semantics. *Journal of Logic and Algebraic Programming*, 60-61, 17-139. <https://doi.org/10.1016/j.jlap.2004.03.009>
- Plotkin, G. D., and Power, J. (2003). Algebraic operations and generic effects. *Applied Categorical Structures*, 11(1), 69-94. <https://doi.org/10.1023/A:1023064908962>
- Plotkin, G. D., and Pretnar, M. (2009). Handlers of algebraic effects. *Proceedings of ESOP 2009*. https://doi.org/10.1007/978-3-642-00590-9_7
- Pnueli, A. (1977). The temporal logic of programs. *Proceedings of FOCS 1977*. <https://doi.org/10.1109/SFCS.1977.32>
- Pnueli, A., Siegel, M., and Singerman, E. (1998). Translation validation. *Proceedings of TACAS 1998*. <https://doi.org/10.1007/BFb0054170>
- Rosu, G., and Chen, F. (2012). Semantics and algorithms for parametric monitoring. *Logical Methods in Computer Science*, 8(1:9). [https://doi.org/10.2168/LMCS-8\(1:9\)2012](https://doi.org/10.2168/LMCS-8(1:9)2012)
- Rosu, G., and Serbanuta, T. F. (2010). An overview of the K semantic framework. *Journal of Logic and Algebraic Programming*, 79(6), 397-434. <https://doi.org/10.1016/j.jlap.2010.03.012>
- Sangiorgi, D. (2011). *Introduction to Bisimulation and Coinduction*. Cambridge University Press. <https://doi.org/10.1017/CBO9780511777110>
- Schneider, F. B. (2000). Enforceable security policies. *ACM Transactions on Information and System Security*, 3(1), 30-50. <https://doi.org/10.1145/353323.353382>
- Strom, R. E., and Yemini, S. (1986). Tpestate: a programming language concept for enhancing software reliability. *IEEE Transactions on Software Engineering*, 12(1), 157-171. <https://doi.org/10.1109/TSE.1986.6312929>
- Van Horn, D., and Might, M. (2010). Abstracting abstract machines. *Proceedings of ICFP 2010*. <https://doi.org/10.1145/1863543.1863553>
- Vardi, M. Y., and Wolper, P. (1986). An automata-theoretic approach to automatic program verification. *Proceedings of LICS 1986*. <https://www.cs.rice.edu/~vardi/papers/lics86rj.pdf>
- Wahbe, R., Lucco, S., Anderson, T. E., and Graham, S. L. (1993). Efficient software-based fault isolation. *Proceedings of SOSP 1993*. <https://doi.org/10.1145/168619.168635>
- Winskel, G. (1986). Event structures. University of Cambridge Computer Laboratory Technical Report UCAM-CL-TR-95. <https://doi.org/10.48456/tr-95>
- Winskel, G. (1993). *The Formal Semantics of Programming Languages: An Introduction*. MIT Press. <https://mitpress.mit.edu/9780262731034/the-formal-semantics-of-programming-languages/>
- Xia, L.-Y., Zakowski, Y., He, P., Hur, C.-K., Malecha, G., Pierce, B. C., and Zdanczewicz, S. (2020). Interaction trees: representing recursive and impure programs in Coq. *Proceedings of the ACM on Programming Languages*, 4(POPL), Article 51. <https://doi.org/10.1145/3371119>